Interview Practice: BA Portfolio Q&A

Kontakt Home - Return Management System Documents

Q1 What personal qualities do you need to become a professional Business Analyst and prepare BA documents like these?

I think there are six important qualities.

First, you need to be curious. A Business Analyst always asks "why?" and "how?" because understanding the real problem is more important than writing documents. If you do not understand the business, your documents will be wrong.

Second, you need to be a good listener. Most of my time goes to talking with people: business owners, developers, testers, and managers. Each of them has different needs, and you need to listen carefully to understand what they really want.

Third, you need to think in a structured way. A BRD document, for example, has many sections: business goals, stakeholders, scope, requirements, and acceptance criteria. You need to organize information in a clear and logical order.

Fourth, you need attention to detail. One missing requirement or one unclear acceptance criterion can cause big problems in development. So you need to check your work many times.

Fifth, you need patience. Writing a good BRD or FRD takes time. You write, you review, you get feedback, and you rewrite. Sometimes you need five or six versions before the document is ready.

And sixth, you need to be able to explain complex things in simple words. Not everyone is technical, so you need to write in a way that a business person can understand and a developer can follow.

Q2 What do you need to know to explain complex things in simple words?

For me, there are three things.

First, you need to understand the business yourself before you write anything. For example, when I wrote the BRD for the Return Management System, I first needed to understand how product returns work in a store: what happens when a customer brings a product back, who checks it, who decides if it can be resold, and how the money goes back to the customer. If you do not understand the real process, you cannot explain it simply.

Second, you need to know your audience. A business manager does not care about API endpoints or database tables. They care about: does this solve my problem? How long will it take? How much will it cost? But a developer does not care about business goals. They need to know: what should I build? What are the rules? What happens if something goes wrong? So you need to write the same thing in two different ways.

Third, you need to know how to use examples and pictures. Instead of writing a long paragraph about how a return works, you can draw a simple flowchart: customer brings product, store checks it, system creates return request, warehouse receives it, system updates inventory. One picture is better than ten paragraphs.

Q3 What was your first task on your first day, and what was the first thing you did?

On my first day, my manager gave me one task: understand the current return process in our stores and find the problems. The company had many complaints from customers about returns, but nobody had a clear picture of the whole process.

The first thing I did was schedule meetings with people who work with returns every day: store managers, warehouse workers, customer support agents, and the finance team. I prepared a simple list of questions: how does a customer start a return? What do you check? What happens to the product? Who approves the refund? How long does it take?

These conversations were the most important step. Before writing any document, I needed to hear the real story from real people. Documents come later, but understanding comes first. That is why the Stakeholder Interview document is the first one in my portfolio, because that is where every project begins.

Q4 You spend hours asking people questions and getting answers. Where and how do you record these answers so you do not lose them? Do you keep them only in your head, or do you write them down? If you write them, do you have a special method?

No, definitely not in my head. If I keep everything in my head, I will forget something. Especially when I talk to five or six different people in one day, each person tells me different things. So I have a simple system.

During the meeting, I take notes by hand in a notebook. I write down the most important points: who I am talking to, their role, what they said, and what problems they mentioned. I do not try to write every word, because if I do that, I stop listening. I only write the key information.

After the meeting, the same day, I organize my notes into a table. I use a simple template with four columns: stakeholder name and role, what they told me, what problems they described, and what they expect from the new system. This table is exactly what you see in the Stakeholder Interview document in my portfolio.

This template is important for three reasons. First, it keeps all information in one place, so I never lose anything. Second, it is easy to share with my manager or the team, because a table is clear and simple to read. And third, when I start writing the BRD later, I already know where to find every piece of information. I do not need to call people again and ask the same questions.

So my rule is simple: write by hand during the meeting, organize into a table the same day, and share with the team before the next meeting. This saves a lot of time and prevents mistakes.

Q5 You said "I only write the key information." Can you explain how you do that? Can you prepare a sample dialogue and show what you write down and what you skip?

Sure. Let me give you a real example. Imagine I am interviewing a store manager about the return process. This is how the dialogue might look.

ME: What happens when a customer brings a product back to your store?

STORE MANAGER: Oh, it is a big headache. Last month we had a lady who bought a washing machine, used it for three weeks, and then said it makes too much noise. But our policy says 14 days for returns. So I had to explain to her that we cannot accept it. She was very angry, called our hotline twice, and even wrote a complaint on Facebook. This happens all the time.

(I write: Return policy = 14 days. Customers often try to return after 14 days. Complaints go to hotline and social media.)

ME: And if the customer is within 14 days, what do you check?

STORE MANAGER: First, I ask for the receipt. If they have the receipt, I check the product box and the accessories. You know, sometimes people return the product but they forget the remote control or the cables inside. Then I call the warehouse guy to come and pick it up. Actually, the

warehouse guy is never on time, he always comes 2-3 hours later, and the customer has to wait. That is another problem.
(I write: Step 1 = check receipt. Step 2 = check product box and accessories. Step 3 = call warehouse for pickup. Problem: warehouse pickup takes 2-3 hours, customer waits.)
ME: Who approves the refund?
STORE MANAGER: I do, but only for products under 200 manat. For more expensive products, I need to call my area manager. She usually answers in one or two hours, but sometimes she is in another store and I cannot reach her. Last week a customer waited four hours for approval.
(I write: Store manager approves refund under 200 AZN. Area manager approves above 200 AZN. Problem: approval can take 1-4 hours depending on availability.)
Now, let me explain what I did in this dialogue.
The store manager talked a lot. She told me a story about a lady with a washing machine. She complained about her warehouse guy. She told me about her area manager. But I did not write: "lady bought washing machine, used it 3 weeks, said it makes noise, was angry, called hotline, wrote on Facebook." That is a story, and the story is not important for my document. What is important is the rule: 14-day return policy. The lady is just an example, but the rule is the key information.
The same with the warehouse guy. I did not write: "he is never on time, he always comes 2-3 hours later." I wrote: "Problem: warehouse pickup takes 2-3 hours, customer waits." I removed the emotion and kept the fact. The fact is useful for the document. The emotion is not.
So my method is: listen to the story, understand the problem, and write down only three things. What is the process step? What is the rule? What is the problem? Everything else, like emotions, names, and long stories, I keep in my head for context, but I do not put it in the document.
Q6 I feel like I understand the Stakeholder Interview document fully now. Are there any more important questions I should ask about it, or can we move on to the next document?
Good question. I think you understand the main idea. But if you want, there are two more things about the Stakeholder Interview that are important.
The first one is: how do you decide who to interview? When I started the return project, I could not talk to everyone in the company. There are hundreds of employees. So I had to choose. My rule was simple: talk to the people who actually do the work and the people who feel the pain. Store managers feel the pain because they deal with angry customers every day. Warehouse workers feel the pain because they receive returned products with missing parts. Customer support feels the pain because they get complaints. And finance feels the pain because they process refunds. Each of these people has a different piece of the picture, and you need all pieces to see the full process.
The second one is: what happens when two people tell you different things? This is very common. For example, the store manager told me: 'the customer gets a refund in one day.' But the finance person told me: 'refund processing takes three to five days.' Both of them are right, but they are talking about different steps. The store manager sees the approval step, and the finance person sees the bank transfer step. As a BA, your job is to combine these different views into one complete picture. That is why in the Stakeholder Interview document, you see different perspectives from different roles, and later in the BRD, you put them all together.
If you have no more questions about the Stakeholder Interview, the next document in my portfolio is the BRD, which is the Business Requirements Document. That is where all the information from the interviews turns into a formal document.
Q7 How do you start preparing the BRD? Do you show the Stakeholder Interview document to someone first, or do you immediately start writing the BRD?

No, I do not start the BRD immediately, and yes, I always show the Stakeholder Interview to someone first. This step is very important.
After I finish all the interviews, I organize my notes and create the Stakeholder Interview document. Then I send it to two groups of people. First, to the stakeholders themselves. I want them to read it and confirm that I understood everything correctly. For example, if the store manager said that refunds take three days, I want her to read my notes and say: 'Yes, that is correct,' or 'No, actually it is five days.' This is called validation, and it prevents big mistakes later.
The second group is my team: my project manager, the lead developer, and the lead tester. They need to read the stakeholder interviews because they will build and test the system. If they see the interviews early, they can start thinking about possible solutions. The developer might read the interview and say: 'If the warehouse pickup takes 2-3 hours, we can build a notification system to tell the customer when the warehouse person is on the way.' So showing the interviews early gives the team time to think.
Only after I get confirmation from stakeholders and feedback from my team, I start writing the BRD. The BRD is not my personal document. It is a team document. The stakeholder interviews are the foundation, and the BRD is the building on top of that foundation. If the foundation is wrong, the building will fall.
So my process is: first, conduct interviews. Second, create the Stakeholder Interview document. Third, validate with stakeholders and share with the team. Fourth, collect feedback. And only fifth, start writing the BRD. This five-step process can take one or two weeks, but it saves months of rework later.
Q8 I understand that you send the Stakeholder Interview to the stakeholders for validation. But what about your team? When they give you feedback, do you add their feedback to the Stakeholder Interview document? If not, where do you keep it? And does the BRD reflect this team feedback?
This is a very practical question, and the answer is: no, I do not add team feedback to the Stakeholder Interview document. These are two different documents with two different purposes. The Stakeholder Interview document has one purpose: capture what the business people said. It is their words, their problems, their needs. If I add the developer's opinion or the tester's comment there, the document becomes mixed, and nobody can tell what the business said and what the team suggested. So the Stakeholder Interview stays clean. It only contains what came out of the interviews.
The team feedback goes to a different place. I keep it in my working notes. These are informal notes that only my team sees. For example, when I send the Stakeholder Interview to the lead developer, he might reply: 'The 2-3 hour warehouse pickup problem can be solved with a simple notification feature. I think we should add that to the scope.' I write this down in my notes, not in the Stakeholder Interview document.
And now, to answer your most important question: yes, the BRD absolutely reflects this team feedback. The BRD is where everything comes together. It contains the business requirements from the stakeholders, the technical suggestions from the team, the constraints from the project manager, and the test considerations from the tester. That is why the BRD is the most important document in the portfolio. It is not a copy of the interviews. It is a smarter, more complete version that combines business needs with technical possibilities.
Let me give you a concrete example. The warehouse manager told me: 'Pickup takes 2-3 hours and the customer waits.' This goes into the Stakeholder Interview. The lead developer told me: 'We can build a notification system for this.' This stays in my working notes. But in the BRD, under the

'Proposed Solution' section, you will see a requirement like: 'The system shall notify the customer when the warehouse person is assigned to the pickup.' So you can see: the business problem from the interview plus the technical solution from the team, together in one requirement inside the BRD.

So the rule is: Stakeholder Interview equals business voice only. Team feedback goes to working notes. BRD equals business voice plus team feedback combined into clear requirements.

Q9 You have the Stakeholder Interview document and the team notes, and you need to create the BRD. What is your plan when you prepare this document? What happens in your mind before you start writing?

Before I write a single word in the BRD, I spend time thinking. This is actually the hardest part of the job, not the writing. Let me explain my thinking process step by step.

First, I read the Stakeholder Interview document one more time, from start to finish. I do not read it like a book. I read it with a pen in my hand. Every time I see a problem or a need, I underline it. For example: 'return policy is 14 days' - underline. 'warehouse pickup takes 2-3 hours' - underline. 'refund approval takes 1-4 hours' - underline. At the end, I have a list of all the problems and needs. This list becomes the raw material for the BRD.

Second, I read my team notes. The developer said we can add a notification system. The tester said we need clear rules for return conditions. The project manager said we have a budget limit and we must finish in two months. I put all of these next to the problems I underlined. So now I have: problem plus technical possibility plus constraint, all next to each other.

Third, I start grouping. This is the most important mental step. I do not write requirements one by one in a random order. I group related things together. For example, everything about the customer return request goes to one group. Everything about warehouse processing goes to another group. Everything about refund and payment goes to a third group. Everything about notifications and communication goes to a fourth group. This grouping helps me see the structure of the BRD before I write it.

Fourth, I think about the structure of the BRD itself. Every BRD has standard sections, and I plan what goes into each section. The 'Business Background' section will explain why we are doing this project, using the problems I underlined. The 'Stakeholders' section will list the people from the interviews and their roles. The 'Scope' section will explain what is included and what is not included in this project. The 'Business Requirements' section will contain the main requirements grouped by topic. The 'Acceptance Criteria' section will explain how we know each requirement is done correctly.

And fifth, only after all this thinking, I open my laptop and start writing. The thinking part takes me one or two days. The writing part takes another two or three days. But because I already have a clear plan in my head, the writing is much faster and much cleaner.

So my mental process is: read and underline problems, connect with team feedback, group related items, plan the structure, and then write. Think first, write second.

Q10 Two questions. First, if your team does not give you any feedback, do you write technical solutions yourself? Second, in the BRD, do you only write the problems, or do you also write the solutions?

Let me answer the second question first, because it will make the first question clearer.

In the BRD, you write both. You write the problem and you write what the business expects as a solution. But there is an important difference between business solution and technical solution. Let me explain with an example.

The warehouse manager told me: 'Pickup takes 2-3 hours, the customer waits, and sometimes the customer leaves before the warehouse person arrives.' In the BRD, I write this as a business requirement: 'The system shall allow the customer to see the real-time status of the warehouse pickup.' This is the business solution. It describes what the business wants to happen. It does not say HOW to build it. It does not say which technology to use. It does not say which database or which programming language. It only says WHAT the system should do.

The technical solution, for example 'we will use WebSocket for real-time updates and PostgreSQL for tracking the pickup status,' is not in the BRD. That level of detail goes into the FRD, the Functional Requirements Document, or the System Design document. The BRD stays at the business level: what problem exists and what result the business expects.

Now, for your first question: if the team does not give me feedback, do I write technical solutions myself? The honest answer is: sometimes yes, but carefully. If I have an engineering background and I can see an obvious solution, I might write it in my working notes and suggest it to the team. But I never put my own technical solution directly into the BRD. Because I am a Business Analyst, not a Solution Architect. My job is to define WHAT the business needs. The team's job is to decide HOW to build it. If I write the technical solution myself in the BRD, two bad things can happen. The developers might feel that I am doing their job, which creates bad relationships. Or my technical idea might be wrong, and the team has to spend time explaining why it will not work.

So my rule is: in the BRD, I always write the business problem and the expected business result. If I have a technical idea, I write it in my working notes and discuss it with the team separately. The team can agree, disagree, or suggest a better solution. But the BRD only contains what the business needs, not how to build it technically.

Q11 Wait, I noticed something confusing. We went directly from the Stakeholder Interview to the BRD. But we did not do a Gap Analysis. In our portfolio, there is a Gap Analysis document. Is this approach correct? Should the Gap Analysis come before the BRD?

This is a very sharp observation. You are right to notice this, and I am glad you caught it. Let me be honest about how it actually works in real projects.

In theory, in a perfect project, the process looks like this: first, stakeholder interviews. Second, Gap Analysis. Third, BRD. The Gap Analysis helps you understand the difference between the current situation, which we call 'as-is,' and the desired future situation, which we call 'to-be.' When you can see the gap clearly, it becomes much easier to write good business requirements in the BRD.

But in real life, especially in small and medium projects, these steps often happen at the same time, not one after another. What I mean is: while I am conducting interviews, I am already thinking about the gaps. When the store manager tells me 'customers wait 2-3 hours for warehouse pickup,' I already see the gap: the current situation is long waiting time, and the desired situation is short waiting time or real-time tracking. So in my head, I am doing the Gap Analysis while I am doing the interviews.

After the interviews, I write both documents almost in parallel. First, I write the Gap Analysis document because it is shorter and simpler. It clearly lists: here is what we have today, here is what we want tomorrow, and here are the gaps between them. This document is very useful because it helps the stakeholders quickly see the big picture. They can look at it and say 'Yes, these are our gaps, this is what we need to fix.' When they approve the Gap Analysis, I feel confident that my understanding is correct.

Only then I start writing the BRD, and the BRD is much better because the gaps are already clear and approved. The business requirements in the BRD are basically the answers to the gaps from the Gap Analysis. For example, if the Gap Analysis says 'Gap: no automated tracking for warehouse

pickup,' then the BRD will say 'Requirement: the system shall provide real-time warehouse pickup tracking.' One gap becomes one requirement.
So to answer your question directly: you are right, the Gap Analysis should come before or together with the BRD, not after. In my explanation, I made it sound like a simple straight line from interviews to BRD, but in reality there is an important step in between. Thank you for noticing this. In an interview, if someone asks you about the process, always remember to mention: interviews, then Gap Analysis, then BRD.

Q12 I did not fully understand this part. There is a current situation that stakeholders tell you about, and a future situation that we want to reach. My first question is: does someone tell you what this future state should be, or do you decide it yourself based on the problems? And if you decide it yourself, how correct is this approach?

This is one of the most important questions you can ask, and the answer is: it is usually a mix of both. Let me explain.
Most of the time, stakeholders tell you their problems, not their solutions. The store manager says: 'Customers wait 2-3 hours for warehouse pickup.' She does not say: 'I want a real-time tracking system with GPS and push notifications.' She only knows her pain. She describes what is wrong today. She does not design the future. That is not her job.
But sometimes, especially the more senior people, they do have a vision. The Head of Retail might say: 'Our goal is that no customer waits more than 30 minutes in the store for any return process.' This is a clear future state. He knows where he wants to go, even if he does not know how to get there technically.
So what do I do when stakeholders only give me problems and not solutions? I do not invent the future state by myself. That would be risky and not correct. Instead, I prepare a proposal. I look at the problems, I talk to my team about what is technically possible, and I write a proposed to-be state. For example, if the problem is long waiting time, my proposed to-be state might be: 'The customer can request a return through the mobile app, the system automatically checks the return eligibility, and the warehouse is notified immediately.'
Then I take this proposal back to the stakeholders and say: 'Based on the problems you described, I think this future state would solve most of them. What do you think?' They can say 'Yes, this is exactly what we need,' or they can say 'No, this is too expensive, we do not need the mobile app part, just make the in-store process faster.' Their feedback changes my proposal, and we work together until we agree on the to-be state.
So the honest answer is: nobody tells me the complete future state, and I also do not decide it alone. I prepare a proposal based on problems, team input, and common sense, and then I validate it with stakeholders. The to-be state is a team decision, not my personal idea. This is why the Gap Analysis is so important. It shows both sides clearly, and everyone can look at it together and agree: 'Yes, this is where we are today, and this is where we want to be tomorrow.' When everyone agrees on both sides, the gap in the middle becomes obvious, and that gap tells you what you need to build.

Q13 After the Stakeholder Interview, do you only prepare the BRD, or do you work on other documents in parallel at the same time?

Good question. In real projects, I never work on just one document alone. Documents are connected, and I prepare them in parallel, not one by one.
When I finish the stakeholder interviews and the team validates them, I usually start three documents at the same time: the Gap Analysis, the BRD, and the User Stories. These three documents feed each other, and working on them together is faster and produces better results.

Let me explain how they connect. The Gap Analysis is the shortest and simplest. I finish it first, in maybe one or two days. It gives me the clear picture: current state, future state, and the gaps. Then I start the BRD, which takes the gaps and turns them into formal business requirements. At the same time, I start writing User Stories. A User Story is a simple way to describe one small piece of functionality from the user's perspective. For example: 'As a customer, I want to see the status of my return request so that I know when to expect the refund.'

Why do I write User Stories in parallel with the BRD? Because User Stories force me to think from the user's point of view. The BRD is a formal document with sections and structures, and sometimes it can become too abstract. But a User Story is always concrete: who is the user, what do they want, and why. When I write User Stories, I often discover that I missed something in the BRD. For example, I might write a User Story about the warehouse worker: 'As a warehouse worker, I want to see the list of pending pickups so that I can plan my route.' And then I realize: I did not include this requirement in the BRD. So I go back and add it.

After these three documents, I usually prepare the Acceptance Criteria for each User Story. The Acceptance Criteria answer one question: how do we know this User Story is done correctly? For example: 'The warehouse worker sees only pickups assigned to him, not all pickups in the city.'

So in the first phase after interviews, I work on Gap Analysis, BRD, User Stories, and Acceptance Criteria, mostly in parallel. They are like a puzzle. Each piece helps you understand the other pieces better. Only after this first phase is complete and approved, I move to the second phase: the more technical documents like the FRD, System Design, REST API specification, and SRS. These documents need the BRD and User Stories as their input.

One more thing: this does not mean I open four Word documents and type in all of them at the same time. That would be impossible. What I do is: I work on one document for a few hours, then switch to another, then come back to the first one. For example, Monday morning I write the Gap Analysis. Monday afternoon I start the BRD. Tuesday I write User Stories. Wednesday I go back to the BRD and improve it based on what I learned from writing the User Stories. This back-and-forth process continues until all documents are complete and consistent with each other.

Q14 Can you explain how you prepare the Gap Analysis document using a concrete example?

Of course. Let me walk you through the real example from our Return Management System project. I will show you exactly how the document looks in my head when I write it.

I start with the first section: Current State, which we also call As-Is. In this section, I describe how returns work today without any digital system. I write it as a simple list, not as a long story. For example:

Current State: Customer comes to the store with the product. Store manager checks the receipt manually. If the product is within 14 days and looks good, the store manager calls the area manager for approval. The area manager takes 1-4 hours to respond. After approval, the store manager fills a paper form. The warehouse person comes to pick up the product in 2-3 hours. The customer waits in the store. After the warehouse receives the product, the finance team processes the refund, which takes 3-5 business days. All tracking is done on paper and phone calls.

Now the second section: Future State, which we call To-Be. This is the proposed solution based on stakeholder problems and team feedback. I write:

Future State: Customer can request a return through the website or mobile app. The system automatically checks if the product is within the return period. The store manager receives the request digitally and can approve or reject it in the system. For expensive products, the area manager gets a notification and can approve with one click. The warehouse receives an automatic

notification when a pickup is needed. The customer can track the return status online. The finance team processes the refund automatically after the warehouse confirms the product condition.
And the third section: the Gaps. This is the most important section. I compare the current state and the future state line by line, and I list what is missing today. For example:
Gap 1: No digital channel for return requests. Customers must come to the store physically.
Gap 2: No automated eligibility check. Store managers check return dates manually.
Gap 3: Slow approval process. Area manager approval takes 1-4 hours by phone.
Gap 4: No real-time tracking. Customers do not know the status of their return.
Gap 5: Paper-based process. All records are on paper, which makes reporting and analytics impossible.
Gap 6: No automated warehouse notification. Warehouse pickup takes 2-3 hours.
These six gaps are the bridge between the current state and the future state. And here is the key point: each gap later becomes one or more requirements in the BRD. Gap 1 becomes the requirement for online return request functionality. Gap 3 becomes the requirement for digital approval workflow. Gap 4 becomes the requirement for status tracking page. And so on.
So the Gap Analysis document has three simple sections: where we are today, where we want to be tomorrow, and what is missing in between. It is usually two or three pages, but it is one of the most useful documents because it gives everyone a clear and simple picture of the project.
Q15 I understand. Can you explain how you write User Stories?
Yes. A User Story has a very simple format. It always follows the same pattern: 'As a [type of user], I want to [do something], so that [I get some benefit].' That is it. Three parts: who, what, and why.
Let me show you with real examples from our Return Management System project.
Example 1: As a customer, I want to submit a return request through the website, so that I do not have to visit the store in person. Here, the user is the customer. The action is submitting a return request online. The benefit is saving time and not going to the store.
Example 2: As a store manager, I want to receive return requests on my dashboard, so that I can review and approve them quickly. The user is the store manager. The action is seeing requests on a dashboard. The benefit is faster processing.
Example 3: As a warehouse worker, I want to see a list of products ready for pickup, so that I can plan my daily route. The user is the warehouse worker. The action is seeing a pickup list. The benefit is route planning.
Now, how do I come up with these User Stories? I go back to two sources. First, the Stakeholder Interview document. Every complaint or need from the interviews can become a User Story.
Second, the Gap Analysis. Each gap can become one or more User Stories. For example, Gap 1 was 'no digital channel for return requests.' That gap directly becomes User Story Example 1. Gap 6 was 'no automated warehouse notification.' That gap becomes User Story Example 3.
But I also think about users that stakeholders did not mention. For example, nobody in the interviews talked about the finance team's needs in detail. But I know that the finance team needs to process refunds, so I write: As a finance officer, I want to see a list of approved returns ready for refund processing, so that I can complete the payments on time.
One important thing: I do not write one big User Story for the whole project. I break it down into small, independent pieces. Each User Story represents one small functionality that can be developed and tested separately. This is important because the development team works in sprints, usually two-week cycles. Each sprint needs a few small User Stories, not one big one.
After I write all the User Stories, I add two more things to each one. First, the Acceptance Criteria. These are the conditions that must be true for the User Story to be considered done. For example,

for the customer return request story, the acceptance criteria would be: the customer must be logged in, the system must check the 14-day return period, the customer must select the product and the reason for return, and the customer must receive a confirmation email after submission.
And second, I add a priority to each User Story. Not all User Stories are equally important. The customer return request is high priority because without it, nothing else works. The warehouse route planning is medium priority because the warehouse can operate without it in the first version. I use the RICE prioritization method for this, which is a separate document in the portfolio. But the simple idea is: high priority stories go into the first sprint, and lower priority stories go into later sprints.
So my process for User Stories is: read interviews and gaps, write one story per small functionality, add acceptance criteria, add priority, and organize them for the development sprints.

Q16 Imagine you are in an interview. The interviewer asks: 'What does a Business Analyst do and why are you good at it?' Can you answer this question using only what we discussed so far, in a natural spoken way?

Of course. Let me try.
A Business Analyst is the person who connects the business side and the technical side of a project. The business people know what problems they have, but they do not know how to describe them in a way that developers can understand. The developers know how to build things, but they do not always understand the business context. The BA sits in the middle and translates between these two worlds.
Let me explain with my last project, the Return Management System. When I joined the project, the company had many customer complaints about returns, but nobody had a clear picture of the whole process. My first task was to understand the current situation. So I scheduled meetings with store managers, warehouse workers, customer support agents, and the finance team. I asked them simple questions: how does a customer start a return? What do you check? What happens to the product? Who approves the refund? How long does it take?
During these interviews, I learned a very important skill: you have to listen to people's stories, but write down only the key information. People tell you stories with emotions and details, but your job is to extract three things from every story: what is the process step, what is the rule, and what is the problem. For example, the store manager told me a long story about an angry customer with a washing machine, but what I wrote down was simply: return policy is 14 days, customers often try to return after 14 days. I keep my interview notes in a structured table with four columns: stakeholder name and role, what they told me, what problems they described, and what they expect from the new system.
After the interviews, I never go directly to writing the BRD. First, I share the interview results with two groups. The stakeholders themselves, to validate that I understood everything correctly. And my team, the project manager, lead developer, and lead tester, because they need to start thinking about solutions early. Their feedback goes into my working notes, not into the interview document. The interview document stays clean with only the business voice.
Then I prepare the Gap Analysis. This is a simple document with three sections: current state, future state, and the gaps. For example, in the current state, warehouse pickup takes 2-3 hours and everything is on paper. In the future state, the warehouse gets automatic notifications and the customer can track the return online. The gaps between them, like 'no digital channel for returns' or 'no real-time tracking,' later become requirements in the BRD. I do not invent the future state alone. I prepare a proposal based on the problems and team feedback, then validate it with the stakeholders until we all agree.

After the Gap Analysis, I work on three documents mostly in parallel: the BRD, the User Stories, and the Acceptance Criteria. The BRD is the main document that contains business problems and expected business results. It describes WHAT the business needs, not HOW to build it technically. The User Stories break the big picture into small pieces, each in a simple format: as a user, I want to do something, so that I get some benefit. And the Acceptance Criteria define how we know each User Story is done correctly.

I think what makes me good at this job is six qualities. You need to be curious and always ask why. You need to be a good listener because most of your time goes to talking with people. You need to think in a structured way because documents like the BRD have many sections that need to be organized logically. You need attention to detail because one missing requirement can cause big problems in development. You need patience because good documents take many versions. And you need to be able to explain complex things in simple words because not everyone is technical.

So in short, my job is to understand the business problems, organize them clearly, and make sure that the right system gets built. Every document I write serves this purpose. The Stakeholder Interview captures the problems. The Gap Analysis shows what is missing. The BRD defines what we need to build. The User Stories break it into small deliverable pieces. And the Acceptance Criteria make sure each piece is done correctly.

Q17 Can you describe that first moment of parallel document preparation so vividly that I feel like I am living through it with you?

Let me take you there. It is a Monday morning, around 9 AM. I am sitting at my desk with a large coffee. On my left side, I have my notebook full of interview notes. On my right side, I have my laptop open with three blank Word documents: Gap Analysis, BRD, and User Stories. On the screen, I also have my working notes open where I wrote the team's feedback from last week.

I start with the Gap Analysis because it is the simplest and it gives me the foundation for everything else. I open a blank page and I write at the top: 'Current State.' Then I close my eyes for a moment and I try to see the whole return process like a movie in my head. I remember the store manager's face when she said 'customers wait 2-3 hours.' I remember the warehouse worker saying 'I never know which product is coming next.' I remember the finance officer saying 'refund takes 3-5 days because everything is on paper.' I start typing. I do not write formally. I write like I am telling a story: 'Customer comes to the store, shows the receipt, store manager checks it, calls area manager, waits 1-4 hours for approval, warehouse comes 2-3 hours later, customer waits.' It takes me about one hour to write the current state. I read it once and I feel good because it matches what I heard in the interviews.

Then I write 'Future State.' I switch to my working notes where the lead developer wrote: 'We can use push notifications for the warehouse.' The project manager wrote: 'Budget is limited, keep it simple.' I combine these inputs with the stakeholder needs and I write the future state: 'Customer requests return online, system checks eligibility automatically, store manager approves digitally, warehouse gets notification, customer tracks status online.' Another hour, and the future state is done.

Now the gaps. This is my favorite part. I print the current state on one page and the future state on another page. I put them side by side on my desk. Literally, two printed papers next to each other. I take a red pen and I draw lines from one page to the other, connecting what exists today with what should exist tomorrow. Each missing connection is a gap. I count them: one, two, three, four, five, six. Six gaps. I type them into the Gap Analysis document. By 12 PM, the Gap Analysis is done. I save it, close it, and I feel relieved because now I have a map.

After lunch, around 1:30 PM, I open the BRD document. The Gap Analysis is still fresh in my mind. I start with the 'Business Background' section. I basically rewrite the current state from the Gap Analysis, but in a more formal tone. Then I write the 'Project Objectives' section, which is basically the gaps rewritten as goals. For example, Gap 4 was 'no real-time tracking,' so the objective becomes 'provide customers with real-time return status tracking.' I work on the BRD for about two hours, then I stop.

Why do I stop? Because something is bothering me. I am writing the BRD and it feels too abstract. I am writing things like 'the system shall support digital return requests.' It sounds good, but I cannot see the actual user. So I switch to the User Stories document. I write: 'As a customer, I want to submit a return request online, so that I do not have to visit the store.' Immediately, the picture becomes clearer. I can see a person sitting at home, opening the website, clicking a button. Then I write another story for the store manager. Then one for the warehouse worker. Each story makes the BRD requirements more concrete in my mind.

Around 4 PM, I go back to the BRD. Now I write better requirements because the User Stories showed me the real users. I add a requirement I missed: 'The warehouse worker shall see only pickups assigned to him.' I got this from the warehouse worker User Story I wrote earlier. This is exactly why I write them in parallel. They help each other.

At 5:30 PM, I stop working. On my screen, I have three documents open. The Gap Analysis is 90 percent done. The BRD is maybe 40 percent done. The User Stories document has about eight stories written. None of them are finished. But I am not worried, because I know that tomorrow I will continue, and the documents will keep helping each other.

Tuesday morning, I open the User Stories first. I write a few more stories for the finance team and the area manager. Then I add Acceptance Criteria to each story: what must be true for this story to be complete. After that, I go back to the BRD and continue where I left off. The Acceptance Criteria I just wrote help me write better requirements in the BRD. By Wednesday, I have a first draft of all three documents. Thursday and Friday, I review them, improve them, and share them with the team for feedback.

So that is how it really looks. Not sitting and writing one document from start to finish. It is more like cooking: you prepare the ingredients, you start cooking one dish, you taste it, you realize you need something from another dish, you switch, you come back, and at the end, all dishes are ready and they taste good together.

Q18 In your last answer, you said that on Tuesday you added Acceptance Criteria to the User Stories. But you did not explain how you actually write the Acceptance Criteria. Can you explain that now?

You are right, I skipped that part. Let me go back to that Tuesday morning and explain exactly how I write Acceptance Criteria.

I have my User Stories document open. Let me pick one story: 'As a customer, I want to submit a return request through the website, so that I do not have to visit the store in person.' This User Story tells me WHAT the customer wants, but it does not tell me the details. The Acceptance Criteria answer the question: how do I test if this story is done correctly? If the developer says 'I finished this story,' how do I verify it?

I think about the happy path first, which means everything works perfectly. Then I think about the edge cases, which means unusual or difficult situations. I write each criterion as a simple statement that can be tested as yes or no.

For this customer return request story, my Acceptance Criteria look like this:

Criterion 1: The customer must be logged in to submit a return request. If the customer is not logged in, the system redirects to the login page. I can test this: log out, try to submit a return, check if the redirect happens.
Criterion 2: The system displays the list of products the customer purchased in the last 14 days. Not all products, only the ones within the return period. I can test this: check if a product bought 20 days ago appears or not.
Criterion 3: The customer must select a product, a reason for return, and optionally add a comment or photo. I can test this: try to submit without selecting a reason, the system should show an error.
Criterion 4: After submission, the system creates a return request with status 'Pending Review' and sends a confirmation email to the customer. I can test this: submit a return, check the database for the status, and check the email inbox.
Criterion 5: The customer can see the submitted return request in their profile page with the return ID, product name, and current status. I can test this: go to profile page after submission and check if the return appears.
Now, I also think about what should NOT happen. This is called negative testing. For example,
Criterion 6: The customer cannot submit a return request for a product that was already returned.
Criterion 7: The customer cannot submit more than 3 return requests per day.
So my process for writing Acceptance Criteria is simple. First, I read the User Story carefully. Second, I think about the happy path: what happens when everything goes right? Third, I think about edge cases: what happens when the user does something wrong, something unusual, or something at the limit? Fourth, I think about what should not happen. Fifth, I write each criterion as a testable yes-or-no statement.
One more thing: the tester on my team loves Acceptance Criteria because they are the basis for writing test cases. Each Acceptance Criterion can become one or more test cases. So when I write good Acceptance Criteria, I am actually helping the tester do their job faster and more accurately. That is why in the parallel preparation process, when I write Acceptance Criteria for User Stories, the whole project becomes stronger: the BRD has clear requirements, the User Stories have concrete descriptions, the Acceptance Criteria have testable conditions, and the tester has a clear starting point for their work.
Q19 I understand. Which documents should we prepare next, after the first phase?
Good question. After the first phase, which includes the Gap Analysis, BRD, User Stories, and Acceptance Criteria, we move to the second phase. This phase has more technical documents that the development team needs to actually build the system.
The first document in the second phase is the FRD, which stands for Functional Requirements Document. If the BRD answers the question 'WHAT does the business need,' the FRD answers the question 'WHAT exactly should the system do.' The FRD takes each business requirement from the BRD and breaks it down into detailed functional requirements. For example, the BRD says 'the system shall support digital return requests.' The FRD explains: what fields must the return form have, what validation rules apply, what happens when the customer submits the form, what status changes occur, and what notifications are sent. The FRD is much more detailed than the BRD.
The second document is the SRS, which stands for Software Requirements Specification. The SRS is similar to the FRD but it also includes non-functional requirements like performance, security, and scalability. For example, the SRS might say 'the system shall process a return request within 3 seconds' or 'the system shall support up to 500 concurrent users.' These are not about what the system does, but about how well the system performs.

The third document is the System Design. This document shows the architecture of the system. It includes diagrams that explain how the different parts of the system connect to each other. For example, it shows that the website sends a request to the API gateway, the API gateway sends it to the return service, the return service checks the database, and the database sends the result back. This document is very important for the developers because it tells them how to structure their code.

The fourth and fifth documents are the REST API Specification and the Swagger OpenAPI Specification. Both documents describe the API endpoints that the frontend and backend use to communicate. The REST API document is simpler and more readable for business people. The Swagger document is in a standard format that developers can import directly into tools like Postman for testing. For example, it defines the endpoint 'POST /api/returns' with all the request parameters, response format, and error codes.

The sixth document is the Postman API Testing document. This is a practical document that contains ready-to-use API tests. The tester or even the BA can open Postman, import this file, and run all the API tests to verify that the backend works correctly. It is a very hands-on document.

Then there are two more documents: the RICE Backlog Prioritization and the SDLC, which stands for Software Development Life Cycle. The RICE Backlog helps the team decide which User Stories to build first and which to build later, based on four factors: Reach, Impact, Confidence, and Effort. The SDLC document describes the overall project plan from start to finish: analysis, design, development, testing, deployment, and monitoring.

And the last document in the portfolio is the UAT Planning document. UAT stands for User Acceptance Testing. This is the final testing phase where the actual business users test the system before it goes live. The UAT document describes who will test, what they will test, and how they will confirm that the system is ready.

So the full picture looks like this. Phase one: Stakeholder Interviews, Gap Analysis, BRD, User Stories, and Acceptance Criteria. These are the business-focused documents. Phase two: FRD, SRS, System Design, REST API, Swagger, Postman Testing, RICE Backlog, SDLC, and UAT. These are the technical and process documents. The first phase feeds the second phase. Without good business documents, the technical documents will be wrong. And without good technical documents, the developers cannot build the system correctly.

Q20 I want to stay in Phase 1 a bit longer. I look at the existing documents and I see a lot of details. I want to understand the chronological order: when exactly does each detail get written? What is the real step-by-step process across all these Phase 1 documents?

I understand your question perfectly. It is one thing to know which documents to write, but it is another thing to know the exact sequence. Let me break it down into very concrete steps, almost like a recipe.

Step 1, which takes maybe 3 to 5 days. I conduct stakeholder interviews. During each interview, I take hand notes. After each interview, the same day, I convert my notes into the structured table format: stakeholder name and role, what they told me, what problems they described, what they expect. By the end of day 5, the Stakeholder Interview document is complete. I send it to stakeholders for validation and to my team for feedback. I wait 2 to 3 days for their responses.

Step 2, which takes 1 to 2 days. While waiting for validation responses, I start the Gap Analysis. I already have all the interview data, so I do not need to wait. I write the Current State section based on interview notes. Then I write the Future State section based on the problems and the team feedback I already have. Then I list the gaps. I told you before that I print both states and put them

side by side with a red pen. That is exactly what happens here. By the end of this step, I have a draft Gap Analysis with maybe 6 to 8 gaps listed.

Step 3, which takes 1 day. Now the validation responses come back. The store manager says: 'Your notes are correct, but you missed one thing: we also check if the product box is damaged.' I go back to the Stakeholder Interview document and add this detail. The lead developer says: 'For Gap 3 about approval speed, we can implement an auto-approval rule for amounts under 50 manat.' I go back to the Gap Analysis and update the Future State with this suggestion. This is the first round of corrections. The documents are getting more accurate.

Step 4, which takes 2 to 3 days. Now I start the BRD. I open a blank BRD document and I go section by section. Business Background: I copy the main problems from the interviews. Project Objectives: I convert the gaps into goals. Scope: I define what is included and what is not. Stakeholders: I list them from the interview document. Business Requirements: this is the main section, and here I spend the most time. I take each gap and I write a formal requirement for it. Gap 1 becomes BR-001. Gap 2 becomes BR-002. And so on. Each requirement gets a unique ID, a title, a description, and a priority. I also add assumptions and constraints that came from the project manager and the team.

Step 5, which takes 1 to 2 days, parallel with Step 4. While writing the BRD, I also write User Stories. For each BRD requirement, I write one or more User Stories. BR-001 about digital return requests becomes three User Stories: one for the customer to submit a return, one for the customer to view return status, and one for the customer to cancel a return. I use the format: as a user, I want to do something, so that I get some benefit. Each User Story gets a unique ID as well: US-001, US-002, and so on.

Step 6, which takes 1 to 2 days, parallel with Steps 4 and 5. While writing User Stories, I immediately write Acceptance Criteria for each story. I do not wait until all User Stories are done. I write US-001, then I write AC-001 through AC-005 for US-001. Then I write US-002, then AC-006 through AC-009 for US-002. I do this because when I write a User Story, the edge cases are fresh in my mind. If I wait two days, I will forget some of them.

Step 7, which takes 1 day. Now I review everything together. I open all four documents: Stakeholder Interviews, Gap Analysis, BRD, and User Stories with Acceptance Criteria. I check for consistency. Is every gap covered in the BRD? Yes. Is every BRD requirement covered by at least one User Story? Yes. Is every User Story covered by Acceptance Criteria? Yes. If I find something missing, I add it. For example, I might realize that the BRD has a requirement about refund processing, but there is no User Story for the finance officer. So I add US-015 for the finance officer and AC-040 through AC-043 for it.

Step 8, which takes 1 to 2 days. I share the BRD, User Stories, and Acceptance Criteria with the team and stakeholders. The lead developer reviews the BRD and says: 'Requirement BR-007 is not clear. What exactly do you mean by real-time tracking?' I rewrite BR-007 with more detail. The tester reviews the Acceptance Criteria and says: 'AC-003 is not testable because it says the system should be fast, but fast is not a number.' I change it to: 'The return request page shall load within 2 seconds.' This is the second round of corrections.

Step 9, which takes 1 day. Final version. After all corrections, I lock all Phase 1 documents. They are now the official baseline for Phase 2. No more changes unless there is a formal change request approved by the project manager.

So if you look at the calendar, the whole Phase 1 takes about 12 to 18 working days, roughly 2.5 to 3.5 weeks. And the key point is: it is not linear. Steps 4, 5, and 6 happen in parallel, not one after

another. I switch between the BRD, User Stories, and Acceptance Criteria many times during those few days. Each document helps improve the others.

Q21 The 9-step overview was still not clear enough for me. Can we go document by document, one at a time, in detail? Starting with the first Phase 1 document?

You are right, let me do that. We already talked a lot about the Stakeholder Interview and the Gap Analysis in our earlier questions. So let me start with the third document: the BRD, the Business Requirements Document. This is the biggest and most important document in Phase 1.

The BRD has several standard sections. Let me go through each section and explain exactly what I write in it and when.

Section 1 is called Document Control. This is the first page of the BRD. I write the document title, the version number which starts at 1.0, the date, the author which is me, and the reviewers which are my project manager and the lead stakeholder. I also write a version history table. Every time someone asks me to change something in the BRD, I increase the version number and write what changed. For example, version 1.1 might say: 'Updated BR-007 based on developer feedback.' This section is small but important because in a real project, the BRD can have 10 or 15 versions, and you need to track what changed in each version.

Section 2 is called Business Background or Introduction. In this section, I write 3 to 4 paragraphs explaining the context of the project. I start with the current situation: 'Kontakt Home is the leading home appliance retailer in Azerbaijan with over 50 stores. Currently, the product return process is fully manual.' Then I explain the problem: 'Store managers receive many customer complaints about long waiting times, lost return forms, and slow refund processing.' Then I explain the business need: 'The company needs a digital Return Management System to improve customer satisfaction and operational efficiency.' I get all this information directly from the Stakeholder Interview document. I basically rewrite the main pain points in a formal tone. I write this section on day 1 of BRD preparation.

Section 3 is called Project Objectives. This is a list of 4 to 6 clear, measurable goals. I convert the gaps from the Gap Analysis into objectives. For example, Gap 1 was 'no digital channel for return requests,' so the objective becomes 'Enable customers to submit return requests online.' Gap 4 was 'no real-time tracking,' so the objective becomes 'Provide real-time return status tracking to customers.' Each objective should be specific enough that you can measure it later. I write this section on day 1 as well, right after the Business Background, because the objectives come directly from the gaps.

Section 4 is called Scope. This section has two parts: In Scope and Out of Scope. In Scope lists everything the project will include. For example: 'Online return request submission, digital approval workflow, warehouse pickup management, refund processing integration, and customer notification system.' Out of Scope lists what the project will NOT include. This is equally important. For example: 'Integration with third-party logistics providers, mobile app development, the project will be web-only in the first version, and automated refund without finance team approval.' The Out of Scope section prevents scope creep, which means people asking for more and more features during the project. When someone says 'Can we also add a mobile app?,' I can point to the BRD and say 'That is Out of Scope for this version.' I write this section on day 2, after I have discussed it with the project manager and the team.

Section 5 is called Stakeholders. This is a simple table with four columns: name, role, responsibility, and contact. I copy most of this from the Stakeholder Interview document. I add the project team members as well: the project manager, the lead developer, the lead tester, and myself as the BA. I write this on day 1 because it is quick and I already have all the information.

Section 6 is the main section called Business Requirements. This is where I spend the most time, usually 2 to 3 days. Each requirement has a unique ID like BR-001, a title, a priority like High or Medium, a description, and sometimes assumptions. Let me give you a real example. BR-001, title: Online Return Request Submission, priority: High. Description: 'The system shall allow registered customers to submit a return request through the Kontakt Home website. The customer shall be able to select a product from their purchase history, specify the reason for return, add optional comments and photos, and receive a confirmation with a unique return ID.' Assumptions: 'The customer has an account on the website. The purchase was made within the last 14 days.' I write maybe 8 to 12 requirements like this, one for each gap and sometimes two for complex gaps.
Section 7 is called Assumptions and Constraints. This is a list of things I assume to be true and things that limit the project. Assumptions: 'All stores have internet connection. The finance team uses the existing ERP system for refund processing.' Constraints: 'The project must be completed within 3 months. The budget does not allow for a mobile app in the first version.' I collect these from the project manager and from the interviews. I write this on day 3.
Section 8 is called Glossary. This is a simple list of terms and their definitions. For example: 'Return Request: a formal request from a customer to return a purchased product.' 'Refund: the process of returning money to the customer for an approved return.' I write this throughout the process and finalize it on the last day. It is especially useful when new team members join the project and need to understand the business language.
So the BRD preparation takes me about 4 to 5 working days for the first draft. Then it goes through review and correction, which adds another 2 to 3 days. The total is about one week to two weeks for a solid BRD. Shall we move to the next document, the User Stories, or do you have questions about any section of the BRD?

Q22 Let's move to the User Stories document. Can you explain each part of it in detail, the same way you did for the BRD?

Of course. The User Stories document is shorter and simpler than the BRD, but it is equally important. Let me explain it in detail.
The User Stories document has a specific structure. Each User Story is like a small card with several fields. Let me list all the fields and explain what goes into each one.
Field 1 is the User Story ID. Just like BRD requirements have BR-001, BR-002, the User Stories have US-001, US-002, and so on. I use sequential numbers. Sometimes I group them by user type. For example, US-001 to US-005 might be customer stories, US-006 to US-009 might be store manager stories, US-010 to US-012 might be warehouse stories. This grouping helps me and the team find stories quickly.
Field 2 is the User Story itself, written in the standard format. 'As a [type of user], I want to [action], so that [benefit].' I already explained this format. But let me add something important: the 'type of user' must be specific. I do not write 'As a user.' I write 'As a registered customer' or 'As a store manager' or 'As a warehouse worker' or 'As a finance officer.' The more specific the user, the clearer the story.
Field 3 is the Epic or Feature group. An Epic is a large group of related User Stories. For example, all stories about submitting and managing returns belong to one Epic called 'Return Request Management.' All stories about warehouse pickup belong to another Epic called 'Warehouse Operations.' All stories about refunds belong to 'Refund Processing.' Grouping stories into Epics helps the product owner and the project manager plan the releases. They can say: 'In Sprint 1, we will do the Return Request Management Epic.' This field is not always required, but in my portfolio, I include it because it shows that I think about the bigger picture, not just individual stories.

Field 4 is the Acceptance Criteria. I already explained in detail how I write these, so I will not repeat. But I want to add one thing: each Acceptance Criterion has its own ID as well, like AC-001, AC-002. And each criterion is linked to the parent User Story. So if US-001 has 5 acceptance criteria, they are AC-001 to AC-005, all under US-001. This linking is important for traceability, which means you can always trace from a criterion back to its story, and from the story back to its BRD requirement.
Field 5 is Priority. I already mentioned RICE prioritization, but let me explain it more concretely. Each User Story gets a priority of High, Medium, or Low. High priority means this story must be in the first sprint because other stories depend on it. For example, the customer login story is High priority because without login, the customer cannot do anything else. Medium priority means it is important but can wait for sprint 2 or 3. For example, the warehouse route planning story is Medium priority because the warehouse can operate without it in the first version. Low priority means it is nice to have and can go to a later release. For example, an advanced analytics dashboard for managers is Low priority for version 1.
Field 6 is Story Points. This is a number that represents how complex the User Story is. The team uses a scale like 1, 2, 3, 5, 8, 13. A story with 1 point is very simple, like changing a button color. A story with 13 points is very complex, like building the entire return request form with validation, file upload, and email notifications. The team estimates the points together in a meeting called planning poker. But as a BA, I give an initial estimate to help the team start the discussion. I base my estimate on how many Acceptance Criteria the story has and how complex they are.
Field 7 is Dependencies. This is a list of other User Stories that must be completed before this one can start. For example, US-003 'Customer views return status' depends on US-001 'Customer submits return request,' because you cannot view a status if you cannot create a return. I also note technical dependencies here. For example, US-006 'Store manager approves return' depends on US-001 because the store manager needs a return request to approve. This field is very important for the development team because it tells them the order in which they must build things.
Field 8 is the BA Notes. This is an informal field where I write any additional context that might help the team. For example, for US-001, I might write: 'The store manager mentioned that customers often forget their order number, so consider adding an option to search by phone number.' This is not a formal requirement, but it is useful context for the developer who will build the feature.
Now, let me walk through one complete User Story example so you can see all fields together. User Story ID: US-001. Epic: Return Request Management. User Story: As a registered customer, I want to submit a return request through the website, so that I do not have to visit the store in person. Acceptance Criteria: AC-001: The customer must be logged in. AC-002: The system shows products purchased in the last 14 days. AC-003: The customer must select a product, a reason, and optionally add a comment and photo. AC-004: After submission, the system creates a return with status Pending Review and sends a confirmation email. AC-005: The customer cannot submit a return for an already returned product. Priority: High. Story Points: 8. Dependencies: None. BA Notes: Consider allowing search by phone number for customers who do not remember their order number.
I prepare the User Stories document over 2 to 3 days. I do not write all stories in one sitting. I write 3 or 4 stories on the first day, then I go back to the BRD and check if I covered all requirements. On the second day, I write 3 or 4 more. On the third day, I add the remaining stories, review dependencies, and finalize priorities. Then I share the document with the team for review, similar to how I share the BRD.
Q23 Let's move to the Acceptance Criteria document. Can you explain each part of it in detail, the same way you did for the BRD and User Stories?

Of course. There is one important thing I need to clarify first. In many projects, the Acceptance Criteria are not a separate document. They are written directly inside the User Stories document, as I showed you in the previous answer. Each User Story has its own Acceptance Criteria listed under it. But in our portfolio, the Acceptance Criteria are in a separate document. Let me explain why, and then I will walk you through the document in detail.

I create a separate Acceptance Criteria document for two reasons. First, the tester on the team needs it as a standalone reference. When the tester writes test cases, they do not want to scroll through 15 User Stories to find all the criteria. They want one clean document where they can see all Acceptance Criteria together, organized by User Story. Second, during UAT, which is User Acceptance Testing at the end of the project, the business users use this document as a checklist. They go through each criterion and say 'Pass' or 'Fail.' A separate document makes this process easier.

Now let me walk through the structure of the Acceptance Criteria document.

The document starts with a summary section. This is a simple table that shows the total numbers. How many User Stories are there in total? How many Acceptance Criteria in total? How many are linked to customer stories, how many to store manager stories, how many to warehouse stories? This summary helps the project manager understand the scope of testing at a glance. For example: 'Total User Stories: 15. Total Acceptance Criteria: 62. Customer stories: 28 criteria. Store manager stories: 18 criteria. Warehouse stories: 10 criteria. Finance stories: 6 criteria.'

After the summary, the document is organized by Epic, the same way the User Stories are grouped. Each Epic is a section header. Under each Epic, the User Stories are listed one by one, and under each User Story, the Acceptance Criteria are listed.

Now, let me explain each Acceptance Criterion in detail. Each criterion has several fields.

Field 1 is the Criterion ID. I use the format AC-001, AC-002, and so on. The numbers are global, not per User Story. This means AC-001 belongs to US-001, AC-002 also belongs to US-001, but AC-006 might belong to US-002. I do this because the tester often refers to criteria by their AC number, and it is easier to say 'AC-015 failed' than to say 'the fourth criterion of User Story US-004 failed.'

Field 2 is the Linked User Story ID. This tells you which User Story this criterion belongs to. For example, AC-001 is linked to US-001. AC-006 is linked to US-002. This is the traceability link I mentioned before. If someone asks 'Why do we have this criterion?' I can trace it back: AC-003 is linked to US-001, which is linked to BR-001 in the BRD. So the full traceability chain is: Acceptance Criterion to User Story to BRD Requirement to Gap in the Gap Analysis to Problem in the Stakeholder Interview. Every criterion has a reason to exist.

Field 3 is the Criterion Description. This is the actual yes-or-no statement. I already gave examples in our earlier conversation. Let me add more detail about how I write these descriptions. I always start with a clear subject. I do not write 'Must be logged in.' I write 'The customer must be logged in to access the return request page.' I do not write 'Check return period.' I write 'The system shall only display products purchased within the last 14 days.' The subject must be clear, the expected behavior must be clear, and the result must be measurable.

Field 4 is the Test Type. Each criterion falls into one of three categories. Positive test: this tests that the system works correctly when everything goes right. For example, 'A logged-in customer can submit a return request successfully.' Negative test: this tests that the system handles errors correctly. For example, 'A customer who is not logged in is redirected to the login page.' Edge case test: this tests unusual or boundary situations. For example, 'A customer tries to return a product purchased exactly 14 days ago, the system shall accept it. A customer tries to return a product

purchased 15 days ago, the system shall reject it.' I mark each criterion with its type so the tester knows what kind of test to write.
Field 5 is the Pre-conditions. This is a list of things that must be true before you can test this criterion. For example, to test 'The system shows products purchased in the last 14 days,' the pre-condition is: 'The customer has at least one purchase within the last 14 days and one purchase older than 14 days.' Without this pre-condition, the tester cannot verify the criterion properly because they have no test data.
Field 6 is the Expected Result. This is what the tester should see when the criterion passes. For example: 'The customer sees a list of products. Products within 14 days are shown with a Return button. Products older than 14 days are shown without a Return button. Each product shows the name, purchase date, and price.' The expected result must be specific enough that the tester does not have to guess. If the expected result says 'The system shows the correct products,' that is too vague. The tester will ask: 'What is correct? Which products? In what format?' So I always write expected results with specific details.
Field 7 is the UAT Status. This field is empty when I first write the document. Later, during UAT, the business users fill it in. They test each criterion and write Pass, Fail, or Blocked. If they write Fail, they also add a comment explaining what went wrong. This field turns the Acceptance Criteria document into a living checklist during the final testing phase.
Let me now show you one complete criterion example with all fields.
Criterion ID: AC-003. Linked User Story: US-001. Test Type: Edge case. Description: 'The system shall only display products purchased within the last 14 days in the return request form.' Pre-conditions: 'The customer account has at least one product purchased 10 days ago and one product purchased 20 days ago.' Expected Result: 'The return request form shows the product purchased 10 days ago with a selectable checkbox. The product purchased 20 days ago is not shown in the list. A message appears saying: Only products purchased within the last 14 days are eligible for return.' UAT Status: empty.
I prepare the Acceptance Criteria document in parallel with the User Stories. I do not wait for all User Stories to be finished. As soon as I finish writing US-001, I immediately write AC-001 through AC-005 for it. Then I write US-002, and immediately AC-006 through AC-009. I do this because when I write a User Story, the edge cases and error scenarios are fresh in my mind. If I wait, I will forget half of them. The whole document takes me about 3 to 4 days to complete.
Q24 In the BRD, you wrote that a mobile app will not be included. How do you make such decisions? Who decides what is in scope and what is out of scope? Can you show me the real process?
This is a very honest question, and I will give you an honest answer. I do not make this decision by myself. The BA does not decide the scope alone. Let me show you exactly how it works.
The scope decision is made in a meeting, usually called a Scope Definition Workshop or a Kickoff Meeting. This meeting happens at the beginning of the project, before I write the BRD. The people in this meeting are: the project sponsor, who is usually a senior manager or director, the project manager, the lead stakeholder, the lead developer, and me as the BA.
The meeting starts with the project sponsor explaining the business goal. For example, the Head of Retail might say: 'We need to digitize the return process. We are losing customers because the current process is too slow. I want to see results within 3 months because the summer sales season is coming.'
Then I present the gaps from my Gap Analysis. I say: 'Based on the interviews, we identified 8 gaps. If we close all 8 gaps, the project will be quite large.' I show them the list and I give a rough

estimate for each gap. For example, I might say: 'Gap 1, digital return channel, this is medium effort. Gap 2, automated eligibility check, this is small effort. Gap 5, paper to digital transition, this is large effort.' I do not give exact numbers because I am a BA, not an estimator. But I give a rough sense of size.

Then the project manager brings the constraints. This is where the reality hits. The project manager says: 'We have a budget of 80,000 manat. We have a team of 5 people. We have 3 months until the summer season. We cannot build everything.'

Now the negotiation begins. The lead developer says: 'If you want a mobile app in addition to the website, that doubles the development time. We would need 6 months, not 3.' The project sponsor thinks for a moment and says: 'The summer season is more important than the mobile app. Let us do the website first and add the mobile app in version 2.'

I ask a clarifying question: 'What about the warehouse route planning feature? Is that critical for version 1?' The warehouse manager says: 'We can survive without it for now. The most important thing is that customers can submit returns online and track the status.' I write this down: warehouse route planning moves to version 2.

Another discussion: the finance team wants automated refund processing. The developer says: 'Automated refund requires integration with the bank, and that integration takes at least 4 weeks. We do not have 4 weeks in our timeline.' The finance officer agrees to keep manual refund processing in version 1, with automation planned for version 2.

At the end of this meeting, usually 2 to 3 hours long, we have a clear list. I write it on a whiteboard or in a shared document. In Scope: online return request, digital approval workflow, customer status tracking, basic refund processing, email notifications. Out of Scope: mobile app, warehouse route planning, automated refund, third-party logistics integration, advanced analytics dashboard. Everyone in the room agrees and signs off. This signed document becomes the basis for the Scope section in my BRD.

So you see, the scope is not my personal decision. It is a team decision based on three factors: business priority, budget and time constraints, and technical feasibility. My role as a BA in this process is to facilitate the discussion, present the gaps, ask the right questions, and document the final decision. I make sure nothing is forgotten and that everyone understands what is in and what is out.

This is also why the Scope section is so important. Three months into the project, someone from marketing might come and say: 'We really need the mobile app for our summer campaign.' I open the BRD, show them the Scope section, and say: 'The mobile app was discussed and agreed as Out of Scope for version 1. If you want to add it now, we need a formal change request, and the project sponsor must approve it.' This prevents the project from growing out of control.

Q25 The kickoff meeting information was very interesting. Can you tell me what other types of meetings or workshops happen in a project? What is the purpose of each one, who attends, and what decisions are made?

Good question. A BA participates in many meetings during a project, and each meeting has a different purpose. Let me list the main ones in chronological order, the same way they happen in real project life.

Meeting number 1: Project Kickoff. We already discussed this one. Purpose: align everyone on the project goals, timeline, budget, and scope. Who attends: project sponsor, project manager, key stakeholders, lead developer, lead tester, and BA. What decisions: scope definition, In Scope and Out of Scope, timeline, budget approval, and team assignments. This meeting usually happens once at the very beginning and takes 2 to 3 hours.

Meeting number 2: Stakeholder Interviews. We discussed this in detail earlier, but let me frame it as a meeting. Purpose: understand the current process, identify problems, and gather requirements. Who attends: the BA and one stakeholder at a time. These are one-on-one meetings. What decisions: no formal decisions are made here. The purpose is information gathering. But indirectly, every problem identified becomes a future requirement. Each interview takes 45 to 60 minutes, and I do 5 to 10 interviews over several days.
Meeting number 3: Requirements Review or Requirements Walkthrough. Purpose: present the BRD and other documents to the stakeholders and the team, collect feedback, and get formal approval. Who attends: project manager, key stakeholders, lead developer, lead tester, and BA. What decisions: stakeholders approve or reject requirements. The developer confirms technical feasibility. The tester confirms that Acceptance Criteria are clear. The project manager confirms that the timeline is realistic. This meeting usually happens once or twice, after the BRD first draft is ready. It takes 1 to 2 hours. After this meeting, I go back and update the documents based on the feedback.
Meeting number 4: Sprint Planning. This is a meeting that happens at the beginning of each sprint, which is usually a 2-week work cycle. Purpose: decide which User Stories the team will work on in the next sprint. Who attends: project manager or scrum master, the whole development team, and the BA. What decisions: which User Stories go into the sprint, how many story points the team can commit to, and who works on which story. The BA's role here is to explain the User Stories and Acceptance Criteria to the developers so they understand what to build. This meeting happens every 2 weeks and takes about 1 hour.
Meeting number 5: Daily Standup. This is a very short meeting that happens every day, usually in the morning. Purpose: each team member answers three questions: what did I do yesterday, what will I do today, and is there anything blocking me? Who attends: the whole team: developers, tester, BA, and project manager. What decisions: no formal decisions. The purpose is synchronization. If a developer is blocked because a requirement is unclear, the BA hears about it immediately and can clarify the same day. This meeting takes only 10 to 15 minutes, and some teams even do it standing up to keep it short.
Meeting number 6: Backlog Refinement or Grooming. Purpose: review and prepare User Stories for future sprints. Who attends: the BA, the project manager, and the lead developer. What decisions: split large User Stories into smaller ones, update priorities, add new stories, remove stories that are no longer relevant, and estimate story points. This meeting happens once per week and takes about 1 hour. The BA brings new or updated stories, and the team discusses and estimates them.
Meeting number 7: UAT Kickoff. This meeting happens near the end of the project, before User Acceptance Testing begins. Purpose: explain to the business users how the testing will work, what they need to test, and how to report bugs. Who attends: the BA, the project manager, the tester, and the business users who will do the testing. What decisions: testing schedule, who tests which module, how to report issues, and what is the go or no-go criteria for the system to go live. The BA explains the Acceptance Criteria document to the business users so they understand what to check. This meeting happens once and takes about 1 to 2 hours.
Meeting number 8: Retrospective. This meeting happens at the end of each sprint or at the end of the project. Purpose: the team discusses what went well, what went wrong, and what to improve next time. Who attends: the whole team. What decisions: action items for process improvement. For example, the team might decide: 'Next time, we want the BA to finish the BRD one week earlier so the developers have more time to ask questions.' This is a learning meeting, and it is very important for the team to grow. It takes about 1 hour.

Meeting number 9: Change Request Review. This meeting happens when someone wants to add, remove, or modify a requirement after the BRD is approved. Purpose: evaluate the impact of the change on the timeline, budget, and scope. Who attends: the BA, the project manager, the lead developer, and the person who requested the change. What decisions: approve the change and extend the timeline, reject the change, or postpone the change to version 2. This meeting can happen at any time during the project when someone requests a change.
So these are the main meetings a BA participates in. Not all of them happen in every project. A small 2-month project might only have kickoff, interviews, one requirements review, and UAT kickoff. A large 6-month project will have all of them. But as a BA, I need to be prepared for any of them because they are all part of the job.
Q26 Can you tell me which Phase 1 document is written for which team member or audience?
Yes, this is a very practical question. Every document has a primary audience, which means the people who use it the most, and a secondary audience, which means the people who also need to read it but use it less. Let me go through each Phase 1 document.
Document 1: Stakeholder Interview. Primary audience: the BA and the project manager. The BA writes it and uses it as the foundation for all other documents. The project manager reads it to understand the business context and the stakeholder expectations. Secondary audience: the stakeholders themselves, because I send it to them for validation. The developers and testers rarely read this document because it contains business language, not technical details.
Document 2: Gap Analysis. Primary audience: the project sponsor and the project manager. The sponsor reads it to understand the big picture: where we are today and where we want to be. The project manager uses it for project planning and scope definition. Secondary audience: the stakeholders, because they need to approve the future state. The lead developer also reads it to understand the business context, but in less detail. The tester usually does not need this document.
Document 3: BRD. This is the document that everyone reads. Primary audience: the project manager, the stakeholders, the lead developer, and the BA. The project manager uses it for overall project planning. The stakeholders use it to verify that their needs are captured correctly. The lead developer uses it as the main input for technical design. The BA uses it as the master reference for all other documents. Secondary audience: the tester, who reads the Business Requirements section to understand what the system should do. Even the project sponsor might read the Business Background and Project Objectives sections to stay informed.
Document 4: User Stories. Primary audience: the development team, which means the developers, the lead developer, and the tester. The developers read User Stories to understand exactly what to build. Each story is small and concrete, which makes it perfect for developers. The tester reads User Stories to understand the expected behavior and to start writing test cases. The lead developer uses them for sprint planning and task assignment. Secondary audience: the project manager and the BA. The project manager uses the priorities and story points for sprint planning. The BA uses User Stories to track project progress during daily standups.
Document 5: Acceptance Criteria. Primary audience: the tester and the business users. The tester uses it as the direct input for writing detailed test cases. Each Acceptance Criterion can become one or more test cases. The business users use it during UAT as a checklist to verify that the system works correctly. Secondary audience: the developers. The developers read the Acceptance Criteria to understand the exact expected behavior, including edge cases and error handling. The BA uses it during testing to verify that the delivered feature matches the original requirement.
Let me also give you a simple summary table format that is easy to remember.
Stakeholder Interview: BA, Project Manager, Stakeholders for validation.

Gap Analysis: Project Sponsor, Project Manager, Stakeholders for approval.
BRD: Everyone. Project Manager, Stakeholders, Lead Developer, BA, Tester.
User Stories: Developers, Tester, Project Manager for sprint planning.
Acceptance Criteria: Tester, Business Users for UAT, Developers for behavior details.
One more important thing: the way I share documents with different audiences is also different. When I send the Stakeholder Interview to stakeholders, I send the whole document. But when the lead developer asks about requirements, I do not send the whole BRD. I send the specific section they need, or I point them to the User Story that is relevant to their current task. This is important because a developer who is building the return request form does not need to read the glossary or the document control section. They need US-001 and its Acceptance Criteria. I save their time by giving them only what they need.

Q27 What is US-001?

US-001 is the unique ID for the first User Story in our Return Management System project. I give every User Story a unique ID so that the team can refer to it easily. US stands for User Story, and 001 means it is the first one.
The content of US-001 is: 'As a registered customer, I want to submit a return request through the website, so that I do not have to visit the store in person.' This is the most important User Story in the whole project because without it, nothing else works. The customer must be able to submit a return online before the store manager can approve it, before the warehouse can pick it up, and before the finance team can process the refund.
It has 5 Acceptance Criteria: AC-001 to AC-005. The customer must be logged in, the system shows eligible products only, the customer must select a product and a reason, after submission the system creates a return with status Pending Review and sends a confirmation email, and the customer cannot return an already returned product.
I used US-001 as an example many times in our conversation because it is the simplest one to understand and it is the first story that the development team would build in Sprint 1.

Q28 You said you do not send the full BRD to the developer, you send US-001. Can you explain what this means in practice?

Yes, let me explain this with a real example. Imagine I am the BA and Elvin is the lead developer on our Return Management System project. It is Monday morning and we are in the Sprint Planning meeting. I tell the team: 'In Sprint 4, we will build US-001, which is the customer return request form, and US-006, which is the store manager approval page.' Elvin looks at me and says: 'OK, I will build US-001. Send me the details.'
Now I have two options. Option 1: I send Elvin the entire BRD document, which is 25 pages long, with document control, business background, scope, stakeholders, 12 requirements, assumptions, constraints, and glossary. Elvin opens the file, he sees 25 pages, and he thinks: 'Where is the part about the return request form? I have to read 25 pages to find what I need?' That is a waste of his time. Developers are expensive and their time is valuable. Every minute they spend reading irrelevant information is a minute they do not spend coding.
Option 2: I send Elvin only what he needs. I send him one email with four things attached. First, US-001 with the full User Story text. Second, the 5 Acceptance Criteria for US-001, which are AC-001 to AC-005. Third, the linked BRD requirement BR-001, but only that one requirement, not the whole BRD. Fourth, if there is a relevant API specification or design diagram, I attach only the page about the return request endpoint.
The email looks like this: 'Hi Elvin, here are the materials for US-001 that you will build in Sprint 4. The User Story describes what the customer needs. The Acceptance Criteria describe the exact

behavior, including edge cases. BR-001 is the parent requirement in the BRD for your reference. Let me know if you have any questions, we can talk after the standup tomorrow.'
Elvin opens the email, he sees 3 pages, not 25. He reads the User Story in 2 minutes. He reads the Acceptance Criteria in 5 minutes. He already knows what to build. He starts coding the same day. If he has a question, he comes to me directly: 'In AC-003, the customer can add a photo. What is the maximum file size?' I check my notes, I tell him: 'The project manager said 5 megabytes per photo.' He continues coding.
Now, the BRD is still important. It is the master document. If Elvin wants to understand why we are building this feature, he can read the Business Background section. If he wants to understand the overall system, he can read all the requirements. But he does not need all of that to build US-001. He only needs the specific story and its criteria.
But there is one exception. When a new developer joins the team in the middle of the project, I do send them the full BRD. Because the new developer needs to understand the big picture before they can work on any specific story. After they read the BRD, then I start sending them specific User Stories, just like I do with Elvin.
So the rule is simple: for the team that is already working on the project, send only what they need for their current task. For new team members, send the full BRD first, then specific stories after.
Q29 Based on all my questions about Phase 1, what important questions do you think I should have asked but did not?
Good question. Let me think about what we covered and what we missed. We discussed the documents, the process, the meetings, and the audience. But there are at least six important questions that an interviewer might ask you, and we have not covered them yet.
Question 1: What happens when two stakeholders disagree? For example, the store manager wants the return process to be as fast as possible with auto-approval for all returns. But the finance officer wants manual approval for every return to prevent fraud. Both are right from their perspective. As a BA, how do you handle this conflict? This is a very common situation in real projects, and an interviewer will almost certainly ask about it.
Question 2: What happens when a developer says your requirement is impossible? Imagine I write BR-007: 'The system shall provide real-time tracking with live location on a map.' The developer looks at it and says: 'We cannot do this in 3 months. Live GPS tracking requires a mobile app, and we agreed the mobile app is Out of Scope.' As a BA, how do you react? Do you change the requirement? Do you argue with the developer? Do you escalate to the project manager?
Question 3: How do you know if your BRD is good or bad? What are the quality criteria? What makes a BRD excellent versus average? An interviewer might ask: 'How do you measure the quality of your own work?' This is a self-reflection question, and it shows whether you have standards.
Question 4: What if you discover a new gap after the BRD is already approved? Imagine we are in Sprint 3 and I suddenly realize that nobody thought about the return of damaged products. The BRD only covers returns of non-damaged products. What do I do? Do I update the BRD? Do I write a new document? Do I wait for the next version?
Question 5: How do you handle a stakeholder who does not want to participate? In real projects, some stakeholders are too busy or simply not interested. The warehouse manager might say: 'I do not have time for interviews, just build the system.' How do you get the information you need without the stakeholder's cooperation?
Question 6: What tools do you use to write these documents? Do you use Microsoft Word, Confluence, Jira, or something else? How do you manage version control? How do you share

documents with the team? This is a practical question that shows whether you have real work experience.
These six questions are important because they test not your document writing skills, but your problem-solving skills, your communication skills, and your ability to handle difficult situations. Any of these could appear in an interview for a BA position. Which one would you like to discuss first?
Q30 What should I do in my first week at Kontakt Home to be successful? How can I show results quickly?
This is a very smart question, because the first week sets the tone for everything. If you start well, people trust you. If you start slowly, it is hard to recover that impression. Let me give you a concrete plan for your first week.
Day 1: Observe and listen. Do not try to change anything on the first day. Your goal is to understand the company culture, the team structure, and the current projects. Ask your manager for the organizational chart. Find out who the key stakeholders are. Ask for access to all existing documentation: current BRDs, project plans, meeting notes, any documentation that exists. Read as much as you can. Take notes. At the end of day 1, you should know: who is on your team, what projects are active, and what the biggest challenges are.
Day 2: Schedule one-on-one meetings with your key stakeholders. Not long meetings, 20 to 30 minutes each. Start with the people you will work with most: your project manager, the lead developer, the lead tester, and the main business stakeholder. In each meeting, ask three questions. First, what are you working on right now? Second, what are the biggest problems you face? Third, what do you expect from the new BA? Write down their answers. These meetings show people that you care about their perspective, which builds trust from day one.
Day 3: Study the existing documents and systems. By now, your manager should have given you access to all project documentation. Read the current BRDs, the backlog, the Jira or Confluence pages. Try to understand the products. If the company has an e-commerce website, go to the website as a customer. Try to place an order, try to return a product, try to use the chat support. Experience the product yourself. If you find problems during this testing, write them down. This shows initiative.
Day 4: Identify one quick win. A quick win is a small improvement that you can deliver in your first week. It should be simple, visible, and useful. For example, you might notice that the existing BRD does not have a glossary section. Add one. Or you might notice that the User Stories in Jira do not have Acceptance Criteria. Write them. Or you might create a simple process flow diagram for a feature that the team is currently building. The key is: do something small but useful that shows your skills immediately. When your manager sees that you already added value in your first week, they will trust you with bigger tasks.
Day 5: Present your findings to your manager. At the end of the first week, schedule a 30-minute meeting with your manager. In this meeting, present three things. First, what you learned about the team, the projects, and the current challenges. Second, the problems or gaps you identified during your document review and product testing. Third, your proposed plan for the next 2 to 4 weeks: which documents you will create or improve, which stakeholder meetings you will schedule, and what you will deliver by the end of month one. This presentation shows that you are organized, proactive, and focused on results.
Now, how do you measure success in that first week? I think there are four simple indicators. First, you know everyone's name and role on your team. Second, you have read all the existing documentation and can talk about it. Third, you have identified at least one gap or problem that nobody else noticed. Fourth, you have delivered at least one quick win, like a glossary, a process

diagram, or updated Acceptance Criteria. If you achieve these four things in your first week, you have started successfully.

One more thing that is very important for Kontakt Home specifically. Since Kontakt Home is an e-commerce company, you should understand their products. Go to the website, browse the product categories, read customer reviews, look at the return policy page. If you understand how the customer experiences the product, you will write much better requirements than a BA who only reads documents and never touches the product. This customer perspective is something that not every BA has, and it will make you stand out.

Q31 I understand the business phase and the technical phase. But what does success really mean? Is it enough for a BA to conduct interviews and write documents?

This is maybe the most important question of our entire conversation. And the honest answer is: no. Writing documents and conducting interviews is only half of the job. If you only do that, you are a document writer, not a Business Analyst. Let me explain the difference.

A document writer finishes the BRD and sends it to the team. Then they wait. They wait for the developer to build it, they wait for the tester to test it, and they wait for the project manager to manage it. The document writer says: 'I did my job, the BRD is ready, it is not my problem anymore.'

A Business Analyst does something completely different. A Business Analyst does not write documents and disappear. A Business Analyst stays with the project from the first interview to the final release and even after that. Let me describe what this looks like in practice.

First, during development, the BA is available every day. The developer starts building US-001, the return request form. On day 2 of development, the developer comes to the BA and says: 'The customer needs to select a return reason, but you listed 8 reasons in the Acceptance Criteria. I think 3 of them are the same. Can we combine them?' The BA does not say: 'The document says 8, build 8.' The BA thinks about it, discusses with the developer, and maybe agrees to combine them. Then the BA updates the Acceptance Criteria. This is flexibility, and it is a critical skill.

Second, during testing, the BA reviews the test results. The tester finds a bug: 'When the customer submits a return for a product that costs more than 500 manat, the system shows an error message that is not clear.' The BA looks at the bug, checks the Acceptance Criteria, and realizes: 'I did not specify what the error message should say for expensive products.' The BA writes a clear error message requirement, updates the AC, and asks the developer to fix it. This is ownership.

Third, the BA communicates with stakeholders throughout the project, not only at the beginning. Every two weeks, the BA sends a short update to the stakeholders: 'Sprint 3 is complete. We finished the return request form and the approval workflow. Next sprint, we will work on warehouse notifications. Here is a screenshot of what the form looks like.' This keeps the stakeholders engaged and informed. Without this communication, stakeholders forget about the project or lose trust in it.

Fourth, the BA thinks about business value, not only about requirements. What I mean is: the BA should always ask 'Is what we are building actually going to solve the problem?' Maybe the team spent 3 weeks building a beautiful return request form, but the real problem was not the form. The real problem was that the warehouse takes 2-3 hours for pickup. A simple notification system would have solved 80 percent of customer complaints, but nobody thought about it because everyone was focused on the form. A good BA steps back and looks at the big picture regularly.

Fifth, the BA helps resolve conflicts. Not just stakeholder conflicts, which we discussed earlier, but also team conflicts. When the developer and the tester disagree about whether a feature works correctly, the BA is the person who reads the Acceptance Criteria together with both of them and

says: 'The criterion says the system shall show an error message within 2 seconds. The current implementation shows it in 4 seconds. So the tester is right, this needs to be fixed.' The BA is the judge, but a fair judge who uses the documents as evidence.
Sixth, the BA participates in the release decision. Before the system goes live, the BA checks: are all high-priority User Stories completed? Are all critical Acceptance Criteria passed? Is the documentation updated? The BA does not just write documents and leave. The BA makes sure the project actually delivers what was promised.
And seventh, after the system goes live, the BA follows up. Two weeks after release, the BA contacts the store managers and asks: 'How is the new return system working? Are customers complaining less? Are there any problems?' If there are problems, the BA documents them and adds them to the backlog for version 2. This is continuous improvement.
So to answer your question directly: success as a BA means you are not just a person who writes documents. You are a person who understands the business, communicates with everyone, solves problems, supports the team, ensures quality, and follows up after delivery. The documents are your tools, but they are not your job. Your job is to make sure the right system gets built and that it actually solves the business problem. That is the difference between a good BA and a great BA.
Q32 I cannot fully internalize the business phase after the interviews. I feel like I understand each document separately, but I cannot see the whole picture. Can you help me build a clear mental model?
I understand. Let me try a different approach. Forget about documents for a moment. Think about a real-life situation that everyone understands: building a house.
Imagine you are not a BA. Imagine you are a person who wants to build a house. You have land, you have money, and you have a dream. But you do not know how to build a house. So what do you do?
Step 1: You talk to your family. Your wife says: 'I want a big kitchen.' Your mother says: 'I want a garden.' Your son says: 'I want a game room.' Your daughter says: 'I want a balcony.' You listen to everyone and you write down what they want. This is the Stakeholder Interview. You are the BA, your family members are the stakeholders, and your notebook is the Stakeholder Interview document.
Step 2: You look at your land and you look at your family's wishes, and you realize: there is a gap. The land is small, but the family wants many things. The current situation: empty land. The future situation: a house with a kitchen, garden, game room, and balcony. The gap: you need to find a way to fit all of this on a small piece of land. This is the Gap Analysis. Current state, future state, and the gaps between them.
Step 3: You go to an architect. The architect asks: 'What do you want?' You do not say: 'Build me a house with a kitchen, garden, game room, and balcony.' That is too vague. You say: 'I want a two-story house. The first floor should have a big kitchen connected to the garden. The second floor should have three bedrooms, one of which can be used as a game room. There should be a balcony on the second floor facing the garden.' This detailed description is the BRD. You took your family's wishes and the gaps, and you turned them into a formal, structured description of what you want to build.
Step 4: The architect says: 'OK, I understand. Let me break this down into smaller tasks.' The architect writes: Task 1: build the foundation. Task 2: build the first floor walls. Task 3: install the kitchen. Task 4: build the stairs. Task 5: build the second floor walls. Task 6: build the game room. Task 7: build the balcony. Each task is small and clear. This is the User Stories. Each User Story is one small piece of the project that can be built and checked separately.

Step 5: For each task, the architect writes specific conditions. For Task 3, install the kitchen: the kitchen must have a window facing the garden, the counter must be at least 3 meters long, there must be space for a refrigerator and a dishwasher, and the water pipes must be connected. For Task 7, build the balcony: the balcony must hold at least 200 kilograms, it must have a safety rail at least 1 meter high, and it must face the garden. These specific conditions are the Acceptance Criteria. They tell you exactly how to check if each task is done correctly.
Now let me map this back to your BA documents.
Stakeholder Interview equals family conversations. The result: a notebook with everyone's wishes.
Gap Analysis equals looking at the land versus the dream. The result: a clear list of what is missing.
BRD equals the formal description you give to the architect. The result: a structured document that says exactly what you want.
User Stories equals the architect's task breakdown. The result: small, buildable pieces of work.
Acceptance Criteria equals the quality conditions for each task. The result: clear pass or fail checks.
And the flow is natural. You cannot write the BRD before you talk to your family, because you do not know what they want. You cannot write User Stories before the BRD, because you do not know what to break down. You cannot write Acceptance Criteria before User Stories, because you do not know what to check. It is a chain. Each step depends on the previous one.
One last thing. In the house example, you do not stop after giving the description to the architect. You visit the construction site every week. You check if the kitchen window really faces the garden. You check if the balcony can really hold 200 kilograms. You talk to the builder when something is unclear. And after the house is finished, you ask your family: 'Are you happy with the kitchen? Is the garden what you imagined?' This is the success part we discussed in the previous answer. The documents are the plan, but the BA stays until the house is built and the family is happy.
Q33 Wait, are you saying that the BRD must be finished before User Stories? I thought you said earlier that you write them in parallel.
You caught a contradiction, and you are right to ask about it. Let me clarify. In my head, the BRD comes first logically, but in practice, I work on them in parallel. Let me explain the difference.
Think about it this way. You cannot write a User Story if you do not know what the requirement is. The User Story US-001 comes from BRD requirement BR-001. So logically, BR-001 must exist before US-001. In that sense, yes, the BRD comes first.
But you do not need the entire BRD to be finished before you start writing User Stories. You only need the first requirement. So what I actually do is: I write BR-001, then I immediately write US-001 for it. Then I write BR-002, and immediately US-002. Then BR-003, and US-003. And so on. I switch back and forth between the two documents. The BRD is always one or two requirements ahead of the User Stories, but I never wait for the entire BRD to be complete.
Let me go back to the house example. The architect writes the first part of the plan: 'The first floor will have a big kitchen.' Then he immediately writes the first task: 'Task 1, build the kitchen foundation.' Then he writes the next part of the plan: 'The first floor will have a living room.' Then he writes the next task: 'Task 2, build the living room walls.' He does not finish the entire plan first and then start writing tasks. He writes a little bit of the plan, then a little bit of the tasks, then a little more of the plan, then a little more of the tasks.
So the correct answer is: the BRD starts first, and the User Stories follow closely behind. They are not completely parallel like two independent tracks. They are more like two people walking together, but the BRD person is always one step ahead. The BRD opens the door, and the User Stories walk through it.

And I do this for a practical reason. When I write BR-001 and then immediately write US-001, the User Story helps me check if the BRD requirement is clear enough. If I cannot write a clear User Story from a BRD requirement, it means the requirement is too vague. For example, if BR-007 says 'The system shall be user-friendly,' I cannot write a User Story for that because 'user-friendly' is not specific. So I go back and rewrite BR-007 to be more specific: 'The system shall allow a customer to submit a return request in fewer than 5 clicks.' Now I can write a User Story for it. The User Story acts as a quality check for the BRD.

So to summarize: the BRD starts first and leads the way. The User Stories follow closely, usually one or two requirements behind. The Acceptance Criteria follow the User Stories in the same way. One leads, the others follow, but nobody waits for the entire previous document to be finished.

Q34 Can you show me the whole Phase 1 process as a live experience? Imagine you just arrived at Kontakt Home and started working on the Return Management System. Make me feel like I am living through it with you.

Let me take you there. It is Monday, your fourth day at Kontakt Home. For the first three days, you met people, read existing documents, and understood the company. Now, on Monday morning, your manager calls you to his office.

'Zamir, we need to digitize the product return process. Customers are complaining, stores are losing time, and everything is on paper. I want you to lead this as the Business Analyst. The project sponsor is the Head of Retail, Elnar. You will work with a team of 4 developers led by Tural, a tester named Aysel, and the project manager is Ramin. Your deadline is 3 months.'

You leave his office. You sit at your desk. You open a new notebook. On the first page, you write: 'Return Management System - BA Notes.' Under that, you write the names: Elnar is the sponsor, Ramin is the PM, Tural is the lead developer, Aysel is the tester. Now what?

You know from experience that you cannot write a single document yet. You need information first. So you open your email and you write to five people: the store manager at the Nizami branch, the warehouse supervisor, the customer support team lead, the finance officer who processes refunds, and the area manager who approves expensive returns. Your email says: 'Hi, I am Zamir, the new BA. I am working on a project to improve the return process. Can I schedule a 45-minute meeting with you this week? I want to understand how returns work from your perspective.'

By the end of Monday, you have 4 meetings scheduled for Tuesday and Wednesday.

Tuesday, 9 AM. You sit in front of the store manager, Leyla. She looks tired. You have your notebook and a pen. You ask your first question: 'Leyla, walk me through what happens when a customer brings a product back to your store.' And Leyla starts talking. She talks for 40 minutes. She tells you about the angry customers, the long waits, the missing receipts, the area manager who never answers the phone, the warehouse guy who is always late. You do not interrupt. You listen. You write. But you do not write everything. You write: 'Return policy 14 days. Receipt required. Check product box and accessories. Area manager approval for products over 200 AZN. Approval takes 1-4 hours. Warehouse pickup 2-3 hours. Customer waits in store.'

Tuesday, 11:30 AM. You meet the warehouse supervisor, Samir. He shows you the warehouse. He points at a corner full of returned products. 'These have been here for a week. Nobody told us what to do with them. We do not have a system, we just write everything on paper.' You write: 'No tracking system. Products sit for days. Warehouse does not know which returns are coming. No digital connection with stores.'

Wednesday morning. Customer support lead, Narmin. She shows you the complaint logs. 'Look, 40 percent of our calls last month were about returns. Customers asking: where is my refund? Why is

it taking so long?' You write: '40% of support calls are return-related. No automated status updates. Customers call repeatedly for updates. Refund processing 3-5 business days.'
Wednesday afternoon. Finance officer, Rasul. He shows you the refund spreadsheet. 'Everything is in Excel. When the warehouse confirms a return, someone emails me, I check the spreadsheet, and I process the refund manually. If I am sick, refunds stop.' You write: 'Refunds processed manually in Excel. No automation. Single point of failure. 3-5 days processing time.'
Wednesday, 5 PM. You go back to your desk. You open your laptop. You create a Word document. At the top, you write: 'Stakeholder Interviews - Return Management System.' Then you create a table with four columns: Name and Role, What They Told Me, Problems They Described, What They Expect. You type everything from your notebook into this table. By 6:30 PM, the Stakeholder Interview document is done. You save it. You lean back. Day 4 complete.
Thursday morning. You email the document to all four people: 'Please review and confirm if I understood everything correctly.' Then you forward it to Ramin, Tural, and Aysel: 'These are the stakeholder inputs. Please review and share any technical suggestions.'
While waiting for responses, you open a new Word document. 'Gap Analysis - Return Management System.' Section 1: Current State. You type a simple description based on your interviews: everything is manual, paper-based, slow, disconnected. 45 minutes. Section 2: Future State. Tural already replied: 'We can build a web-based return portal. Notifications are easy with our current email service.' You combine stakeholder expectations with Tural's input. Another 45 minutes. Section 3: Gaps. You list them: no digital channel, no automated eligibility check, slow manual approval, no customer tracking, paper-based records, no warehouse notification. Six gaps. It is 1 PM. Gap Analysis draft done.
Thursday afternoon. Responses arrive. Leyla says: 'You forgot to mention we also check the product box for physical damage.' You update the Stakeholder Interview. Tural says: 'For Gap 3, we can implement auto-approval for returns under 100 AZN.' You update the Gap Analysis future state. Ramin says: 'Budget is 80,000 AZN. Timeline strict, 3 months.' You note this for the BRD scope section.
Friday morning. New Word document. 'Business Requirements Document - Version 1.0.' Document control: author Zamir, reviewers Elnar and Ramin. Business Background: you rewrite the stakeholder complaints in formal language. Project Objectives: you take each of the 6 gaps and turn it into a goal. Scope section: In Scope and Out of Scope based on Ramin's constraints. Stakeholders: names from the interview document. This takes until lunch.
Friday afternoon. The main section: Business Requirements. BR-001: Online Return Request Submission. Priority: High. BR-002: Automated Eligibility Check. BR-003: Digital Approval Workflow. BR-004: Customer Return Status Tracking. BR-005: Warehouse Pickup Notification. By 4 PM, you have 6 requirements. The BRD is maybe 40 percent done. You save and close.
Monday, week 2. You continue the BRD. BR-007, BR-008. Assumptions and Constraints. Glossary. First draft complete. You send it to Ramin, Elnar, and Tural.
While the BRD is under review, you start User Stories. BR-001 becomes US-001: 'As a registered customer, I want to submit a return request through the website.' Immediately, the Acceptance Criteria: AC-001: customer must be logged in. AC-002: system shows products within 14 days. AC-003: customer selects product, reason, optional photo. AC-004: system creates return with status Pending Review. AC-005: cannot return already-returned product. You write US-002, US-003, and their criteria. By Wednesday, 8 User Stories with 35 Acceptance Criteria.
Thursday. Review responses arrive. Ramin: 'BR-007 about real-time tracking is not clear. What does real-time mean?' You rewrite BR-007. Tural: 'Notifications in BR-005 should include SMS, not just

email.' You update BR-005. Aysel: 'AC-003 is not testable. What is the maximum photo file size?' You update AC-003 with specific limits.
Friday, end of week 2. All Phase 1 documents complete: Stakeholder Interviews, Gap Analysis, BRD, User Stories, Acceptance Criteria. Ramin confirms: 'These look good. Sprint 1 starts next Monday.' You lean back. Two weeks. Five documents. One clear picture of what needs to be built.
That is the real experience. Not sitting and writing documents one after another. Talking to people, listening carefully, thinking, writing simply, checking, fixing, and making sure everyone agrees before the developers start building.
Q35 I learned many new things. Now can you explain in more detail how you make decisions while writing the documents? What goes through your mind at each decision point?
You are right, I mostly showed you what I write and when I write it. But I did not show you the thinking behind the writing. Let me do that now. I will go through each document and show you the key decisions I make and why.
Decision 1: Who to interview. When my manager says 'digitize the return process,' my first decision is: who do I talk to? I do not go to everyone. I think about the process flow. A customer brings a product, the store receives it, someone approves it, someone picks it up, someone processes the refund. Each step has a person. The store manager handles the receiving. The area manager handles the approval. The warehouse worker handles the pickup. The finance officer handles the refund. So I need these four people. But I also add the customer support lead because she hears all the complaints, which gives me a different perspective. My decision rule is: talk to the people who touch the process at each step, plus the people who hear about the problems. That is how I chose the five interviewees.
Decision 2: What to write in the interview notes. During the interview with Leyla, she talks for 40 minutes. I cannot write 40 minutes of speech. I need to decide what to keep and what to drop. My decision rule is: keep only three things. Process steps, business rules, and pain points. Process steps: customer arrives, checks receipt, checks product, calls area manager. Business rules: 14-day policy, 200 AZN approval threshold. Pain points: 2-3 hour wait, area manager not answering. Everything else, like emotions, personal stories, and complaints about other departments, I keep in my head for context but I do not write it in the document.
Decision 3: How to structure the Current State. When I write the Gap Analysis Current State, I need to decide how to organize it. I could write it as a long story, or I could write it as a numbered list, or I could write it as a process flow. I choose a simple narrative with key facts. Why? Because the audience is the project sponsor and the stakeholders. They are business people, not technical people. A long story will bore them. A numbered list is too dry. A simple narrative with facts is the easiest to read and understand. The format decision depends on the audience.
Decision 4: How to define the Future State. This is the hardest decision. I cannot invent the future state alone. But I also cannot ask the stakeholders to design it, because they are not technical. So I make a middle-ground decision: I take the stakeholder wishes, add the team's technical suggestions, and combine them into a proposed future state. Then I present it for validation. The key word is proposed. I never present it as final. I always say: 'This is my proposal based on what I heard. Please tell me if this matches your vision.' This protects me. If they say no, I change it. If they say yes, we move forward together.
Decision 5: How many gaps to identify. After I write the Current State and Future State, I need to identify the gaps. The question is: how detailed should I be? Should I write 4 gaps or 8 gaps or 15 gaps? My decision rule is: each gap should be large enough to become a separate feature or requirement, but small enough to be explained in one sentence. If a gap needs two sentences to

explain, it might be two gaps. If two gaps are almost the same, they should be one. For example, 'no digital channel' and 'no online form' are the same gap. I combine them into one: 'no digital channel for return requests.' I ended up with 6 gaps for this project, which felt right. Not too many, not too few.

Decision 6: BRD requirement granularity. When I write the Business Requirements, I need to decide how detailed each requirement should be. If I write too much detail, the BRD becomes a technical document, which it should not be. If I write too little, the developers will not understand what to build. My decision rule is: each requirement should answer three questions. What should the system do? Who benefits from it? What is the business rule or constraint? For example, BR-001: 'The system shall allow registered customers to submit return requests through the website.' What: submit return requests. Who: registered customers. Rule: through the website. That is enough detail for the BRD. The technical details go into the FRD later.

Decision 7: How many User Stories per requirement. When I break down BRD requirements into User Stories, I need to decide how many stories to write. BR-001 is about online return requests. Should this be one User Story or five? My decision rule is: one User Story per user action. The customer can submit a return, view return status, and cancel a return. These are three different actions, so they become three User Stories: US-001, US-002, US-003. I do not combine them into one big story because the development team needs small, deliverable pieces. But I also do not split them into too many tiny stories, because that creates overhead. Usually 2 to 4 User Stories per BRD requirement is the right balance.

Decision 8: Acceptance Criteria depth. For each User Story, I need to decide how many Acceptance Criteria to write. US-001 has 5 criteria. Should it have 3 or 10? My decision rule is: cover the happy path, cover the main error cases, and cover the most important edge cases. That is it. I do not try to cover every possible situation. If I write 15 criteria for one story, the developer will be overwhelmed and the tester will spend too much time. 4 to 7 criteria per story is usually the right range. More than 10 means the story is too large and should be split into two stories.

Decision 9: Priority assignment. Every User Story needs a priority. How do I decide what is High, Medium, or Low? My decision rule is based on three factors. Dependency: if other stories depend on this one, it is High. Business impact: if the sponsor specifically asked for this feature, it is High. Complexity: if a story is simple and can be built quickly, it might be better to do it early even if it is not the most important feature. For example, US-001 is High because it is the foundation, everything depends on it. US-010 about warehouse route planning is Medium because it is useful but not critical for version 1. US-015 about advanced analytics is Low because it is nice to have.

Decision 10: When to stop writing and start reviewing. This is a decision every BA struggles with. When is a document good enough? My decision rule is: a document is ready for review when I can read it from start to finish without stopping and thinking 'this part is unclear.' If I myself get confused reading my own document, it is not ready. I also check three things before sending for review. First, every gap from the Gap Analysis must be covered in the BRD. Second, every BRD requirement must have at least one User Story. Third, every User Story must have Acceptance Criteria. If these three checks pass, I send the documents. I do not try to make them perfect, because perfect documents do not exist. Good enough documents that get reviewed and improved are better than perfect documents that never get shared.

So these are the 10 main decisions I make during Phase 1. Each decision has a simple rule behind it. The rules come from experience and from thinking about what the audience needs. As you do more projects, these decisions become faster and more natural. In your first project, you will spend

a lot of time thinking about these decisions. In your third project, you will make them almost automatically.

Q36 Can we move to Phase 2? Before we start talking about Phase 2 documents, what do I need to know as a BA? What knowledge or concepts should I understand first?

Yes, we can move to Phase 2. But before we talk about the documents, you need to understand some important concepts. Phase 2 is different from Phase 1. In Phase 1, you talk to business people and write business documents. In Phase 2, you work closer with the technical team and write documents that help developers build the system. So let me explain the key things you need to know before we start.

Concept 1: The Difference Between Business Requirements and Functional Requirements. In Phase 1, you write Business Requirements in the BRD. A business requirement says what the business needs. For example, BR-001 says: 'The system shall allow customers to submit return requests online.' This is a business requirement. It says WHAT the system should do, but it does not say HOW the system will do it. In Phase 2, you write Functional Requirements in the FRD. A functional requirement explains HOW the system will do it. For example: 'When a customer clicks Submit Return Request, the system shall validate the product serial number against the purchase database, check the 14-day return window, and display a confirmation message with a return ID.' See the difference? The BRD tells everyone what needs to happen. The FRD tells the developer exactly what to build. As a BA, you need to understand this difference clearly because you write both documents.

Concept 2: What an API Is and Why It Matters. In Phase 2, the development team starts building the system. The system has a front-end, which is what the customer sees on the website, and a back-end, which is the server and the database. The front-end and the back-end need to talk to each other. They talk through APIs, which stands for Application Programming Interface. Think of it like a waiter in a restaurant. The customer sits at the table and gives an order to the waiter. The waiter takes the order to the kitchen. The kitchen prepares the food and gives it back to the waiter. The waiter brings the food to the customer. The waiter is the API. The customer is the front-end. The kitchen is the back-end. You do not need to know how to code an API. But you need to understand what it is because you will write API documentation and you will review it with the developers. When a developer says 'I need a POST endpoint for return requests,' you should understand that the front-end is sending data to the back-end to create something new.

Concept 3: What REST API Means. REST is a style for building APIs. It is the most common style used in web development. REST uses standard HTTP methods. There are four main methods you need to know. GET means read data. For example, GET /api/returns returns a list of all returns. POST means create new data. For example, POST /api/returns creates a new return request. PUT means update existing data. For example, PUT /api/returns/123 updates return number 123. DELETE means remove data. For example, DELETE /api/returns/123 deletes return number 123. As a BA, you do not write code. But when you write the FRD or the API documentation, you need to understand these four methods because each functional requirement connects to one or more API endpoints. If a requirement says 'the system shall display return status,' the developer will create a GET endpoint. If a requirement says 'the customer shall submit a return,' the developer will create a POST endpoint.

Concept 4: What Swagger and Postman Are. Swagger is a tool that generates API documentation automatically from code. The developer writes special comments in the code, and Swagger reads

those comments and creates a web page that shows all the API endpoints, what data they accept, and what they return. As a BA, you will review the Swagger documentation to make sure it matches the FRD. If the FRD says the system returns a return ID, you check Swagger to confirm that the API response includes a return ID field. Postman is a tool for testing APIs manually. Before the front-end is built, the developer can use Postman to send requests to the back-end and see the responses. As a BA, you might use Postman during UAT testing to test individual API endpoints. You do not need to be an expert in these tools, but you need to know what they are and what they are used for.

Concept 5: What the RICE Framework Is. When you have many features to build but limited time and money, you need to decide which features to build first. The RICE framework helps you make this decision. RICE stands for Reach, Impact, Confidence, and Effort. Reach is how many users will use this feature. Impact is how much it helps them. Confidence is how sure you are about your estimates. Effort is how many weeks or sprints it will take. The formula is simple: Reach times Impact times Confidence divided by Effort. The higher the score, the higher the priority. For example, if Online Return Requests has high reach, high impact, high confidence, and medium effort, it will have a high RICE score and will be built first. Advanced Analytics has low reach, low impact, medium confidence, and high effort, so it will have a low score and will be built later. As a BA, you will help the team calculate RICE scores and organize the backlog.

Concept 6: What SDLC Means. SDLC stands for Software Development Life Cycle. It is the process that the development team follows from the start of the project to the end. There are several SDLC models, but the most common one in modern companies, including Kontakt Home, is Agile. In Agile, the project is divided into short cycles called sprints. Each sprint is usually 2 weeks long. At the start of each sprint, the team picks the highest priority items from the backlog and commits to completing them by the end of the sprint. During the sprint, there is a daily 15-minute meeting called a standup where each person answers three questions: What did I do yesterday? What will I do today? Is anything blocking me? At the end of the sprint, the team demonstrates what they built and does a retrospective to discuss what went well and what can be improved. As a BA, you participate in sprint planning, daily standups, and demonstrations. You also write new user stories and acceptance criteria for future sprints based on what you learned during the current sprint.

Concept 7: What UAT Is and Why You Lead It. UAT stands for User Acceptance Testing. This is the final testing phase before the system goes live. In UAT, the actual business users test the system to confirm it works the way they expect. This is different from the tester's job. The tester tests for bugs and technical errors. The business users test for usability and business correctness. For example, the tester checks if the Submit Return button works without crashing. The business user checks if the return process feels natural and if all the business rules are applied correctly, like the 14-day policy. As a BA, you lead UAT. You prepare test scenarios based on the Acceptance Criteria. You invite the business users. You guide them through the test scenarios. You collect their feedback. You report the results to the team. If something does not work as expected, you write a defect report and the team fixes it. After all critical defects are fixed, you give the final sign-off that the system is ready for production.

These are the 7 key concepts you need to understand before we start talking about Phase 2 documents. You do not need to be a technical expert. You need to understand these concepts at a high level so you can communicate with the development team effectively and write documents that

are useful for everyone. When you are ready, tell me, and we will start with the Phase 2 documents one by one.

Q37 You explained the Phase 2 documents and technical concepts. But I want to understand something different. Besides the technical knowledge, what else do I need to know to work effectively with the technical team? What skills, behaviors, or mindsets should I have when I sit with developers, testers, and the project manager every day?

This is a very good question. Technical knowledge is important, but it is not enough. Many BAs know the technical concepts but still fail because they do not know how to work with people. In Phase 2, you spend most of your time with the technical team, not with business stakeholders. The way you communicate, the way you handle disagreements, and the way you present yourself make a huge difference. Let me share the most important things I have learned.

Skill 1: Learn to Speak Two Languages. In Phase 1, you speak with business people. You say things like 'customer satisfaction will improve' and 'return processing time will decrease.' In Phase 2, you sit with developers and testers. They speak a different language. They say things like 'the endpoint returns a 200 status code' and 'the database query takes 3 seconds.' As a BA, you need to understand both languages and translate between them. When a developer says 'I cannot implement real-time tracking because we do not have WebSocket support,' you need to translate this to the stakeholder: 'The system can show updates every 5 minutes instead of instantly, which still solves the problem.' And when the stakeholder says 'I need it instantly,' you translate back to the developer: 'Is WebSocket something we can add in Sprint 3?' You are the bridge. If you cannot translate, communication breaks down and the project suffers.

Skill 2: Build Trust With Developers Before You Need It. On your first day in Phase 2, do not start by telling developers what to do. Start by listening. Ask them questions about their work, their experience, and their opinions about the project. Show them that you respect their expertise. Say things like 'Tural, I know you have more technical experience than me. I want to understand your perspective on this requirement before I finalize it.' When developers feel respected, they share more information with you. They will tell you about potential problems early, before those problems become big issues. But if you act like you know everything, they will stop sharing information with you, and you will discover problems too late. I learned this the hard way in my first project. I thought being confident meant having all the answers. I was wrong. Being confident means asking good questions and admitting when you do not know something.

Skill 3: Know How to Handle 'This Is Impossible.' Developers will sometimes tell you that a requirement cannot be done. This happens in almost every project. When a developer says 'this is impossible,' do not argue. Do not say 'but the stakeholder wants it.' Instead, do three things. First, ask why. 'Tural, can you help me understand what makes this difficult? Is it a technology limitation, a time constraint, or a cost issue?' The reason matters because different reasons have different solutions. If it is a technology limitation, maybe there is an alternative approach. If it is a time constraint, maybe we can do a simpler version first and improve it later. If it is a cost issue, maybe we can discuss it with the PM. Second, ask for alternatives. 'If we cannot do exactly this, what is the closest we can do that still solves the business problem?' Developers are problem solvers. When you give them a problem instead of a fixed requirement, they often find creative solutions. Third, never make promises to stakeholders without checking with the team first. If the developer says no, you go back to the

stakeholder and say 'I discussed this with the team, and there is a technical limitation. Here are two alternative options. Which one do you prefer?' Always bring solutions, not problems.

Skill 4: Learn to Say 'I Do Not Know, But I Will Find Out.' In Phase 2, people will ask you questions you cannot answer. The developer might ask 'What is the maximum file size for return photos?' The tester might ask 'Should the system allow returns without a receipt if the customer has a loyalty card?' The PM might ask 'Can we fit this feature into Sprint 2 or does it need Sprint 3?' You will not always know the answer. And that is completely fine. What matters is what you do next. Never guess. Never make up an answer. Say 'I do not know, but I will find out and get back to you by end of day.' Then go find the answer. Check the BRD, check with the stakeholder, check with the team. And then follow up. People respect a BA who says 'I do not know' more than a BA who gives wrong answers confidently. This is a skill that takes practice. In the beginning, you might feel embarrassed to say 'I do not know.' But remember, nobody expects you to know everything. They expect you to be honest and reliable.

Skill 5: Prepare Before Every Meeting. In Phase 2, you have many meetings: sprint planning, backlog refinement, daily standup, and technical discussions. Never go to any meeting without preparation. Before sprint planning, review the backlog and know which stories are ready for the sprint. Before backlog refinement, read the stories you plan to discuss and think about questions the team might ask. Before a technical discussion, review the relevant requirements and write down the key points you want to cover. When you come prepared, the meeting is productive and people notice. When you come unprepared, you waste everyone's time and you lose credibility. My rule is simple: spend 15 minutes preparing for every 30-minute meeting. It makes a big difference. One practical tip: write your meeting notes before the meeting, not during. Before the meeting, write down the 3-5 key points you want to discuss. During the meeting, you just check them off and add notes. This keeps the meeting focused and productive.

Skill 6: Manage Changes Without Losing Trust. During Phase 2, stakeholders will change their minds. A stakeholder might say 'I know we agreed on 14 days for the return policy, but the marketing team wants to change it to 30 days.' This happens all the time. When a change request comes, do two things. First, write it down formally. Create a small document or an email that says: 'Change Request: Return policy period changed from 14 days to 30 days. Requested by: Elnar. Date: today. Impact: affects US-001, US-002, AC-003, AC-007. Estimated effort: 2 additional developer days.' This formal record protects you and the team. Second, discuss the impact with the PM and the lead developer before agreeing. Say 'I received a change request. Let me discuss the impact with the team before I confirm.' Then go to Tural and Ramin and say 'Elnar wants to change the return period from 14 to 30 days. How does this affect our sprint plan?' They will tell you if it is possible, how much extra time it needs, and what the risks are. Then you go back to Elnar with facts: 'The change is possible, but it will add 2 days of work and may delay Sprint 2 by 1 day. Do you want to proceed?' This way, stakeholders make informed decisions, and the team does not feel like changes are forced on them without discussion.

Skill 7: Understand the Difference Between Your Job and the Developer's Job. This is very important. Your job is to define WHAT the system should do. The developer's job is to decide HOW the system will do it. Do not cross this line. For example, if the FRD says 'The system shall send an email notification when a return is approved,' your job is done. You do not tell the developer to use

SendGrid or SMTP or any specific email service. That is the developer's decision. Similarly, the developer should not change your requirements without discussing with you first. If Tural decides to send SMS instead of email, he should come to you and say 'I think SMS is better than email for this notification. Can we discuss?' And then you discuss it together. When everyone respects each other's responsibilities, the project runs smoothly.

Skill 8: Be Visible Every Day. In Phase 2, you should be present in the team's daily life. Join the daily standup. Sit near the team if possible. Eat lunch with them sometimes. Ask how their weekend was. This is not about being social. This is about building trust and staying informed. When you are physically or virtually present every day, you hear about problems early. Someone might mention in a casual conversation 'I am stuck on the return status API' and you can help by clarifying the requirement. If you were not there, that person might stay stuck for two days before telling anyone. Also, when the team sees you every day, they feel that you are part of the team, not someone who just sends documents by email. This changes how they communicate with you and how seriously they take your input.

Skill 9: Track Everything and Follow Up. In Phase 2, you will have dozens of small tasks every day. Review this document. Update that requirement. Check if AC-005 is testable. Follow up on the change request from last week. Confirm the sprint scope with the PM. If you try to remember everything in your head, you will forget things. Use a simple task tracker. It can be a notebook, a spreadsheet, or a tool like Jira. Write down every action item, who is responsible, and the deadline. At the end of each day, review your list and check what you completed and what you need to follow up on tomorrow. This habit is simple but powerful. People notice when you always follow up. They trust you more because they know nothing falls through the cracks. And you feel more organized and less stressed because you are not trying to remember everything.

Skill 10: Know When to Push Back. Sometimes a stakeholder will ask for something that is not good for the project. For example, 'I want to add a feature that lets customers negotiate the refund amount.' As a BA, you should evaluate this request carefully. Does it solve a real problem? Is it technically feasible? Does it fit within the timeline and budget? If the answer to any of these questions is no, you need to push back. But pushing back is a skill. Do not say 'that is a bad idea.' Instead, say 'Let me understand the business value of this feature. Who would use it and how often? Let me also check the technical feasibility with the team.' Then you come back with facts: 'I discussed this with the team. The negotiation feature would require significant additional development time, and based on our data, less than 2 percent of returns involve any dispute about the refund amount. I recommend we add it to the Phase 2 backlog for future consideration instead of the current release.' This approach shows that you thought about it seriously, you checked with the team, and you have a data-driven recommendation. Stakeholders respect this much more than a simple yes or no.

These are the 10 skills that I believe make the difference between an average BA and a good BA in Phase 2. Technical knowledge gets you started. These skills keep you effective and respected throughout the project.

Q38 In a project with dozens of small pieces, what is the most effective method to manage everything? How do you avoid forgetting things? What should I keep in my memory and what should I keep in tools? How can I answer any project question instantly, even when there are 40 or 50 different components?

This is one of the most important questions you can ask. Because the truth is, no BA can remember 50 requirements, 200 acceptance criteria, 15 stakeholder names, 8 meeting types, and all the technical details in their head. Nobody can. The secret is not about having a better memory. The secret is about having a better system. Let me explain my complete system for managing a large project without forgetting anything.

Part 1: What Goes Where (The Memory Pyramid).

The first rule is: your brain is for thinking, not for storing. You should not try to remember details. You should remember patterns and connections. Here is how I divide information into three levels.

Level 1: In Your Brain (Things You Must Remember Without Looking). There are only a few things you should keep in your head at all times. First, the project name and the main goal in one sentence. For our project: 'Digitize the return process to reduce customer complaints and processing time.' Second, the 5-6 key stakeholder names and their roles. Elnar is the sponsor, Ramin is the PM, Tural is the lead developer, Aysel is the tester, Leyla is the store manager, Narmin is customer support. Third, the 6-8 main BRD requirement numbers and their titles. BR-001 is online return request, BR-002 is eligibility check, BR-003 is approval workflow, BR-004 is status tracking, BR-005 is warehouse notification. You do not need to remember every detail of each requirement. You just need to know the number and the title. Fourth, the current sprint number and what is being built. 'We are in Sprint 3 and building the approval workflow.' Fifth, the top 3 risks or open issues. 'The notification service integration is delayed, the design team has not finished the mobile layout, and Elnar wants to change the return policy from 14 to 30 days.' That is it. About 20-30 pieces of information. If someone asks you a question about the project, you start from these anchors and then look up the details.

Level 2: Quick Reference (Things You Can Find in Under 30 Seconds). These are the things you keep organized so you can find them immediately but you do not memorize them. The acceptance criteria details, the specific business rules, the stakeholder contact information, the meeting notes, the API endpoint details, and the change request history. The key tool for this level is a well-organized folder structure and consistent file naming. I will explain this in Part 3.

Level 3: Deep Storage (Things You Look Up When Needed). These are the full documents, the design specifications, the test reports, the email chains, and the historical decisions. You do not need these during a conversation. You need them when you are writing documents, preparing for meetings, or investigating a problem. The key for this level is good search capability. When someone asks a question and you say 'let me check and get back to you,' this is where you look.

Part 2: The Daily Review System (How to Never Forget).

The biggest reason BAs forget things is not that they have bad memory. It is that they write things down once and never look at them again. Information that you review regularly stays in your memory. Information that you never review disappears. Here is my review system.

Every morning, before you start working, spend 10 minutes doing what I call the 'Project Scan.' Open your project tracker and read through it. Not every detail, but the summary. Read the sprint status. Read the open issues. Read the upcoming deadlines. This 10-minute scan refreshes your memory and

prepares you for the day. When the PM asks you in the elevator 'How is the return project going?' you can answer immediately because you just reviewed it 30 minutes ago.

Every Friday, spend 30 minutes doing a 'Weekly Review.' This is deeper. Read through all the changes that happened during the week. Update your project tracker. Check if any action items are overdue. Write a short summary of the week: what was completed, what was delayed, what is planned for next week. This weekly review is your safety net. If something slipped through the cracks during the week, the weekly review catches it.

Before every stakeholder meeting, spend 15 minutes reviewing the relevant documents. Before meeting Elnar, review the BRD and the sprint progress. Before meeting Tural, review the FRD and the current user stories. This preparation means you go into every meeting with the right information fresh in your mind.

Part 3: The Tool System (How to Organize Everything).

The tools you use matter less than how you organize them. But here is the system that works best for me.

Tool 1: The Project Tracker (Your Command Center). This is the single most important document you create. It is not a formal project document. It is your personal workspace. I use a spreadsheet for this. It has 5 tabs. Tab 1: Requirements Summary. One row per BRD requirement. Columns: BR Number, Title, Priority, Status (Not Started, In Progress, Testing, Done), Assigned Sprint, Developer, Notes. This is your helicopter view of the entire project. When someone asks 'how many requirements are done?', you open this tab and count. Tab 2: Sprint Board. Columns: Sprint Number, Story Numbers, Status, Blockers, Expected Completion. This tells you what is happening right now. Tab 3: Action Items. Columns: Date, Description, Owner, Deadline, Status. Every time someone asks you to do something or you need to follow up on something, it goes here. Tab 4: Decisions Log. Columns: Date, Decision, Requested By, Discussed With, Rationale. This is critical. Every time the team makes a decision, write it down. Who requested it, who was in the room, and why the decision was made. Three months later, when someone asks 'why did we choose email instead of SMS?', you open this tab and you have the answer. Tab 5: Risk Register. Columns: Risk Description, Impact (High/Medium/Low), Probability (High/Medium/Low), Mitigation Plan, Owner. This helps you track what could go wrong and what you are doing about it.

Tool 2: The BRD-Requirements Traceability Matrix. This is a formal document but it serves a practical purpose. It is a table that connects every BRD requirement to its User Stories, Acceptance Criteria, API Endpoints, and Test Cases. One row per requirement. When someone asks 'which API endpoints are affected if we change the return policy?' you open this matrix, find BR-001, and see all the connected items. Without this matrix, you would have to search through 5 different documents to find the answer. With this matrix, it takes 30 seconds.

Tool 3: The Meeting Notes Template. Use the same template for every meeting. Date, Attendees, Agenda Items, Discussion Notes, Decisions Made, Action Items. Consistency is key. When all your meeting notes look the same, you can find information quickly. When every meeting note has a different format, you waste time trying to understand the structure before you can find the content.

Tool 4: The Stakeholder Map. A simple document with stakeholder names, roles, contact information, their interests, their concerns, and their decision-making power. Before any meeting, check this document to remind yourself who you are talking to and what they care about. If you are meeting Elnar, you know she cares about budget and timeline. If you are meeting Tural, you know he cares about technical feasibility and clean architecture.

Part 4: The Question-Answer Technique (How to Answer Instantly).

Now, the core of your question: how to answer project questions instantly when there are dozens of components. Here is my technique.

Step 1: Anchor. When someone asks you a question, start by connecting it to something you know from Level 1 (your brain). For example, if someone asks 'What happens when a customer submits a return for a product that costs 300 AZN?', you anchor to BR-001 (Online Return Request) and BR-003 (Approval Workflow). You say: 'Based on BR-001, the customer submits the request, and based on BR-003, since the amount is over 200 AZN, it requires area manager approval.' You answered the question using the numbers and titles you keep in your head.

Step 2: Redirect if needed. If the question goes deeper than what you keep in your brain, do not guess. Say: 'Let me give you the high-level answer now, and I will send you the detailed answer by email in 30 minutes.' Then you look up the details from your project tracker or the relevant document. This is much better than guessing or saying 'I do not know.' You gave a partial answer immediately, which shows you understand the project, and you committed to a follow-up, which shows you are reliable.

Step 3: Use Numbers. When you answer questions, always use reference numbers. Say 'According to AC-003, the photo size limit is 5 MB' instead of 'I think the photo limit is 5 MB.' Numbers show that your answers are based on documents, not on memory. People trust numbered answers more than 'I think' answers. This also makes it easy for others to verify your answer by checking the document themselves.

Step 4: Build a Mental Map. After working on a project for 2-3 weeks, you will naturally build a mental map of how everything connects. BR-001 connects to US-001 through US-003, which connect to AC-001 through AC-012, which connect to 4 API endpoints, which connect to 3 database tables. You do not need to memorize this map. It forms naturally as you work with the documents every day. The daily review system I described earlier accelerates this process. Because you review the project every morning, the connections form faster and stronger.

Part 5: The Memory Training Method.

Here is a practical exercise you can do to build your project memory. Every evening, after work, close your laptop and try to answer these 10 questions from memory: What is the project name and goal? Who are the key stakeholders? What are the BRD requirement numbers and titles? What sprint are we in? What stories are being developed right now? What are the top 3 open issues? What decisions were made this week? What meetings are scheduled for tomorrow? What is the next deadline? What

did I learn today? If you can answer all 10 from memory, your project knowledge is strong. If you cannot answer 3 or more, you need to review your project tracker the next morning. Do this for 2 weeks and you will see a dramatic improvement. The first week will be hard. The second week will be easier. By the third week, you will answer most questions without thinking.

This is the complete system. It is not about having a super memory. It is about having a smart system that works with your brain, not against it. Tools store the details. Your brain stores the patterns. Reviews keep everything fresh. And the question-answer technique makes you look confident and professional even when you do not have every detail memorized.

Q39 In Phase 1, you explained to me step by step how each business document is created: interviews first, then gap analysis, then BRD, then user stories, then acceptance criteria. You explained why this order, what each document is based on, how one feeds the next. Now do the same thing for Phase 2. Explain each technical document one by one: what it is, how it is created, what it is based on, why it comes in this order, and how one document feeds the next. Make me understand the full chain from SRS to UAT.

Great question. In Phase 1, I showed you how business documents connect to each other like a chain. Phase 2 is the same idea, but now the documents are for the technical team, not for business stakeholders. The audience changes. The language changes. But the principle is the same: each document is based on the previous one, and each document feeds the next one.

Let me explain all 9 Phase 2 documents one by one, in the exact order I would create them, and show you the full chain.

Document 1: SRS - Software Requirements Specification

The SRS is the bridge between business and technology. In Phase 1, I wrote the BRD for business people. The BRD says what the business needs, in business language. The SRS takes the approved BRD and translates it into system requirements. It says what the software must do, in language that both business and technical people can understand.

Think about it like this. The BRD says: "The customer must be able to submit a return request online." The SRS says: "The system shall provide a web form where registered customers can create a return request by selecting an order, choosing items, specifying the reason, and uploading a photo." See the difference? The BRD states the business need. The SRS states what the system must do to satisfy that need.

How do I create it? I take the approved BRD, open it on my left screen, and create the SRS on my right screen. For each BRD requirement, I write one or more system requirements. BR-001 about online return submission becomes SR-001, SR-002, SR-003. Each system requirement is specific, testable, and unambiguous. I also add non-functional requirements: performance, security, usability. The BRD does not usually cover these, but the SRS must.

What is it based on? The approved BRD from Phase 1. Without the BRD, I cannot write the SRS, because I do not know what the business needs.

Why is it first? Because every other technical document depends on it. The FRD depends on the SRS. The System Design depends on the SRS. The API Specification depends on the SRS. If the SRS is wrong, everything after it will be wrong too. This is why I spend extra time making sure the SRS is accurate.

Document 2: FRD - Functional Requirements Document

The FRD takes the SRS and adds detailed functional behavior. The SRS tells you what the system must do at a high level. The FRD tells you exactly how the system behaves in every scenario: what

happens when the user clicks submit, what happens when the input is invalid, what calculations the system performs, what data is stored where.
For example, the SRS says: "The system shall validate return eligibility." The FRD says: "When a customer selects items for return, the system checks: the order was delivered within the last 14 days, the item category is returnable, the customer has not exceeded the maximum return limit of 3 per month, and the item is not in the non-returnable list. If any condition fails, the system displays an error message with the specific reason. If all conditions pass, the system displays the estimated refund amount based on the original payment method minus any applicable restocking fee."
How do I create it? I sit with the lead developer, Tural, and go through each system requirement from the SRS. I ask him: "For SR-001, when the customer submits a return request, what exactly should happen step by step?" Tural explains the logic. I write it down in a structured format: input, processing, output, business rules, error handling.
What is it based on? The SRS. Each functional requirement maps back to one or more system requirements.
Why does it come second? Because you need to know what the system must do before you can define how it behaves in detail. The SRS gives you the what. The FRD gives you the how. And you need the what before the how.
Document 3: System Design
The System Design document is where the technical team decides how the software will be built. It covers the architecture: what components the system has, how they connect to each other, what database to use, what technology stack, what the data flow looks like, how security is implemented.
This is the most technical document in the Phase 2 chain. The BA does not write this document alone. The BA writes the requirements section and the business context. The system architect or lead developer writes the technical architecture, the database schema, the API design, and the infrastructure decisions. The BA facilitates and coordinates, but the developers are the authors of the technical sections.
How is it created? After the FRD is approved, I schedule a design workshop with Tural and the development team. I present the functional requirements. Then I ask them: "Based on these requirements, what is the best architecture? What database structure do we need? What third-party integrations are required?" They discuss, debate, and decide. I document their decisions and organize them into the System Design document.
What is it based on? The FRD. The developers need to know the detailed functional behavior before they can design the system architecture. You cannot design a house until you know how many rooms it needs, where the doors go, and what the plumbing requirements are.
Why does it come third? Because the architecture must support the functional requirements. If you design the architecture first and then discover new functional requirements, you may need to redo the entire design. The correct order is: know what the system must do (SRS), know exactly how it behaves (FRD), and then design the architecture to support all of that (System Design).
Document 4: REST API JSON Specification
Now we get to the API layer. The REST API JSON Specification defines every API endpoint that the system exposes. Each endpoint is documented with its URL, HTTP method, request parameters, request body format, response body format, error codes, and examples.
For the Return Management System, this means documenting endpoints like: POST /api/v1/returns (create return request), GET /api/v1/returns/{id} (get return details), PUT

/api/v1/returns/{id}/status (update return status), GET /api/v1/returns/eligibility (check return eligibility), and so on.
How do I create it? After the System Design is approved and the team has decided on the API architecture, I take the FRD and the System Design, and for each functional requirement, I define the corresponding API endpoint. I think about it from the frontend perspective: if the mobile app needs to display return requests, there must be a GET endpoint. If the customer needs to create a return, there must be a POST endpoint.
For each endpoint, I write the JSON structure for the request and response. This is detailed work. Every field name, data type, required/optional status, and validation rule must be specified. I also write example JSON payloads so the frontend developer can see exactly what the data looks like.
What is it based on? The FRD and the System Design. The FRD tells me what data the system needs to send and receive. The System Design tells me the API architecture and the URL patterns.
Why does it come fourth? Because you need the system architecture to know what APIs to define. If the System Design says the system uses a microservices architecture, the APIs will be organized differently than if it uses a monolithic architecture. The API specification must follow the architecture.
Document 5: Swagger / OpenAPI Specification
The Swagger or OpenAPI Specification is a formal, standardized version of the API documentation. It is not a different document from the REST API JSON Specification. It is the same information, but in a format that machines can read.
Why do we need both? The REST API JSON Specification is written in a human-friendly format. I write it as a Word document with tables and examples. Any BA or developer can read it. But a machine cannot read it. Swagger and OpenAPI solve this problem. They use a standard format, either JSON or YAML, that tools can parse.
What does this give you? First, developers can import the Swagger file into their IDE and get auto-completion for API calls. Second, testers can use tools like Postman to auto-generate test cases. Third, frontend developers can use tools to generate client code automatically. Fourth, you can host the Swagger file on a server and get an interactive API documentation page where anyone can try the endpoints.
How do I create it? I take the REST API JSON Specification and convert each endpoint into the OpenAPI format. Each endpoint becomes a path object with operation details. I use the OpenAPI 3.0 specification format. This is technical, but once you understand the structure, it is mechanical work.
What is it based on? The REST API JSON Specification. It is the machine-readable version of the same document.
Why does it come fifth? Because it depends on the API specification being complete. You cannot create a Swagger file if you do not know all the endpoints, parameters, and response formats.
Document 6: Postman API Testing
The Postman API Testing document takes the Swagger specification and creates actual test cases for each API endpoint. This is where the BA works with the QA team. For each endpoint, we define test cases: what request to send, what response to expect, what status code to check, and what edge cases to cover.
For example, for the POST /api/v1/returns endpoint, the test cases include: valid return request with all required fields, return request with missing product ID, return request for non-returnable item, return request after the 14-day deadline, return request with invalid order number, and

many more. Each test case has a pre-condition, the request payload, the expected response, and the expected status code.
How do I create it? I set up Postman, import the Swagger file, and for each endpoint, I create a collection of test cases. I write the test scripts that validate the response. For example, the test script checks: status code is 201, response body contains return ID, refund amount matches the calculated amount. I also write the test data: sample products, sample orders, sample customers.
What is it based on? The Swagger specification and the FRD. The Swagger gives me the endpoint details. The FRD gives me the business rules and validation logic that the tests need to verify.
Why does it come sixth? Because you need the API specification complete before you can test it. And you need the business rules from the FRD to know what test cases to write.
Document 7: RICE Backlog Prioritization
The RICE Backlog Prioritization document takes all the functional requirements and user stories and prioritizes them using the RICE framework: Reach times Impact times Confidence, divided by Effort. This tells the team what to build first and what to build later.
How do I create it? I take every user story from Phase 1 and every functional requirement from Phase 2. For each item, I estimate four numbers. Reach: how many users or transactions are affected per month. Impact: how much this feature improves the business, on a scale of 1 to 5. Confidence: how confident I am in my estimates, on a scale of 1 to 100 percent. Effort: how many developer-weeks this will take. Then I calculate the RICE score and sort everything from highest to lowest.
For example, US-001 (online return request submission) might score: Reach 5000 customers per month, Impact 4, Confidence 80 percent, Effort 3 weeks. RICE score equals 5000 times 4 times 0.8, divided by 3, which equals 5333. This is a high score, so it goes into Sprint 1. A feature like SMS notifications might score lower and go into Sprint 3.
What is it based on? The User Stories from Phase 1, the FRD from Phase 2, and business input from stakeholders.
Why does it come here in the chain? Because I need all the requirements to be defined before I can prioritize them. In practice, I start the RICE prioritization early, but I finalize it after the FRD because new functional requirements may emerge during FRD writing that change the priorities.
Document 8: SDLC - Software Development Life Cycle
The SDLC document defines how the project will be executed. It is the project plan for the development team. It covers the methodology (Agile/Scrum), sprint duration, roles and responsibilities, meeting schedule, tools, and the overall timeline from sprint planning to deployment.
How do I create it? I work with the project manager and the Scrum Master. We decide on sprint length (usually 2 weeks). We decide on the ceremony schedule: Sprint Planning on Monday, Daily Standup every morning, Sprint Review on Friday of the last sprint day, Retrospective after the review. We define the roles: Product Owner, Scrum Master, Development Team, BA, QA. We also define the tools: Jira for task tracking, Confluence for documentation, Bitbucket for code, Postman for API testing, Swagger for API documentation.
What is it based on? The project scope from the BRD, the RICE priorities, and the company's existing development process.
Why does it come here? Because you need to know what you are building (all previous documents) before you can plan how to build it. The SDLC is the operational plan that ties everything together and tells the team when and how they will deliver.
Document 9: UAT Planning and Coordination

The UAT document is the last one in the chain because User Acceptance Testing happens at the end of the development cycle. But the planning for UAT starts during development. This document defines who will test the system, what they will test, when they will test it, and what criteria must be met for the system to pass UAT.
How do I create it? I take the Acceptance Criteria from Phase 1 and the functional requirements from the FRD. For each user story, I create a UAT test scenario. The test scenario describes what the tester does, what they expect to see, and how they record the result. I also define the UAT environment: is it a staging environment that mirrors production? Is it a separate test environment? Who provides the test data? What happens when a defect is found?
I also define the UAT participants. These are not QA engineers. These are the actual business users: store managers, warehouse supervisors, customer support agents, finance officers. They are the people who will use the system in real life. Their job during UAT is to say: "Yes, this works the way I need it to work" or "No, this does not match what I need."
What is it based on? The Acceptance Criteria, the FRD, and stakeholder input. The testers are the same stakeholders I interviewed in Phase 1.
Why is it last? Because UAT happens after the developers have built the system. You cannot plan the details of UAT until you know exactly what the system does, which is defined in all the previous documents. But I start writing the high-level UAT plan early, during the FRD phase, so that the stakeholders know testing is coming and can block time in their calendars.
Now let me show you the full chain. This is the complete flow from Phase 1 to Phase 2:
Phase 1 (Business): Stakeholder Interviews gives input to Gap Analysis. Gap Analysis gives input to BRD. BRD gives input to User Stories. User Stories give input to Acceptance Criteria.
Phase 2 (Technical): BRD gives input to SRS. SRS gives input to FRD. FRD gives input to System Design. System Design gives input to REST API JSON Specification. REST API JSON Specification gives input to Swagger/OpenAPI. Swagger/OpenAPI gives input to Postman API Testing. FRD and User Stories give input to RICE Backlog. BRD, FRD, and RICE give input to SDLC. Acceptance Criteria and FRD give input to UAT Planning.
This is a chain. Each link depends on the previous link. If one link breaks, the chain breaks. This is why I spend so much time on each document. Not because I love writing documents. But because I know that every document I write now saves hours of confusion later.
One more important thing. In real life, you do not wait for one document to be 100 percent finished before starting the next one. Just like in Phase 1, where I started User Stories before the BRD was finished, in Phase 2, I start the FRD before the SRS is fully approved, I start the API specification while the System Design is being reviewed, and I start UAT planning while development is still ongoing. The documents overlap. But the logical dependency chain stays the same. You cannot create something that depends on something that does not exist yet.
This is the Phase 2 process. Nine documents, one chain, from business requirements to user acceptance testing. Each document has a clear purpose, a clear audience, and a clear dependency on the previous document. When you understand this chain, you understand how a BA works in a software project.
Q40 You created a Phase 1 Decision Rules document for me. Can you create the same type of document for Phase 2? I want the same format: decision rules for each technical document, a timeline, and a quick memory checklist. This will help me remember the key rules when I create SRS, FRD, System Design, API specifications, Postman tests, RICE backlog, SDLC, and UAT.
Absolutely. I created the BA Phase 2 Decision Rules Quick Reference document for you. It follows the exact same format as the Phase 1 version, but covers all technical documents.

Here is what the document contains. It has 20 decision rules organized into 9 sections, matching each Phase 2 document.
Section A covers SRS Writing Rules with 3 rules. Rule 1 is about SRS granularity: the developer must know what to build but still decide how. Not too vague, not too detailed. Rule 2 is about non-functional requirements: cover at least 5 categories including performance, security, usability, reliability, and compatibility. Rule 3 is about traceability: every system requirement maps back to a BRD requirement, no orphans allowed.
Section B covers FRD Writing Rules with 3 rules. Rule 4 is about scenario structure: every functional requirement gets a happy path, a main error, and an edge case. Rule 5 is about business logic depth: write every formula, every validation rule, every conditional branch. The developer does not know your business rules. Rule 6 is about error handling: at least 2-3 error cases per requirement, including the error message, HTTP status code, and data behavior.
Section C covers System Design Rules with 2 rules. Rule 7 is about the BA role: facilitate and document, do not make architecture decisions. You ask the questions, the developer answers, you write it down. Rule 8 is about design decision documentation: every decision needs four elements, what, why, who, and what requirement. No undocumented decisions.
Section D covers REST API Specification Rules with 2 rules. Rule 9 is about endpoint identification: walk through every user story, each data need becomes a GET, each user action becomes a POST, PUT, or DELETE. Rule 10 is about JSON consistency: camelCase fields, plural resources, UUID IDs, ISO 8601 dates.
Section E covers Swagger and OpenAPI Rules with 2 rules. Rule 11 is about standards: use OpenAPI 3.0 or later, include info, servers, paths, and components sections. Rule 12 is about API versioning: version in the URL path, not in headers, keep old versions running for 6 months.
Section F covers Postman API Testing Rules with 2 rules. Rule 13 is about test coverage: at least 5 tests per endpoint, one valid, two invalid inputs, one auth failure, one edge case. Rule 14 is about environments: set up Local, Staging, and Production environments with setup requests for test data.
Section G covers RICE Backlog Rules with 2 rules. Rule 15 is about estimation approach: BA estimates Reach and Impact, developer estimates Effort, Confidence is a joint discussion. Rule 16 is about sprint allocation: fill sprints to 80 percent capacity, reserve 20 percent for bugs and unplanned work.
Section H covers SDLC Rules with 2 rules. Rule 17 is about ceremonies: 2-week sprints, Planning Monday morning, Daily Standup 15 minutes max, Review and Retro on sprint end Friday. Rule 18 is about tools: use company tools, set up traceability in Jira linking stories to requirements and test cases.
Section I covers UAT Planning Rules with 2 rules. Rule 19 is about tester selection: invite the same stakeholders from Phase 1 interviews, at least one tester per user group. Rule 20 is about pass/fail criteria: all Critical and High defects fixed, 90 percent of scenarios pass, all user groups signed off.
After the 20 rules, the document has two more sections. Part J is the Phase 2 Timeline, a table showing what happens each week from Week 1 to Week 5. Part K is the Quick Memory Checklist with 20 items, one for each rule, written as short reminders you can scan before an interview.
The file is saved as BA_Phase_2_Decision_Rules_Quick_Reference.docx in your portfolio. Together with the Phase 1 Decision Rules, you now have a complete decision-making framework covering all 30 documents across both phases.
Q41 I have a question about Jira tasks. When I create tasks in Jira, should the task names be the same as the names in the documents? For example, a developer gets a task to create a database

table or a Java class. How should Jira tasks be created? What should the task title be, what should the description contain? Is there a standard way to create Jira tasks?

This is a very practical question. Every BA works with Jira, and every BA struggles with this at first. Let me explain the complete approach to creating Jira tasks.

First, let me answer your main question: should the Jira task name be the same as the document name? The answer is no. They serve different purposes. The document name describes the business requirement. The Jira task name describes the work the developer needs to do. These are different things.

Let me give you an example to make this clear. In the BRD, you have BR-001: "Online Return Request Submission." In the User Stories, you have US-001: "As a registered customer, I want to submit a return request through the website." Now, in Jira, the Epic is called "Return Management System." The Story in Jira has the title: "Implement return request submission form." Inside that story, there are sub-tasks. One sub-task says: "Create database table for return requests." Another sub-task says: "Build REST API endpoint POST /api/v1/returns." Another says: "Design return request form UI component." Notice the difference. The BRD talks about business needs. The Jira tasks talk about technical work.

Now, let me explain how to create Jira tasks properly. There are four levels in Jira that you need to understand.

Level 1: Epic. The Epic is the big container. It represents the entire project or a major feature. For the Return Management System, the Epic name is simply "Return Management System - Phase 1." The Epic has a description that summarizes the project scope, taken from the BRD executive summary.

Level 2: Story. Each User Story from your User Stories document becomes a Jira Story. The Story title is not the User Story text. The Story title is a short, actionable summary. The full User Story text goes into the Story description. So the title is: "Implement return request submission." The description contains the full User Story: "As a registered customer, I want to submit a return request through the website, so that I do not need to visit the store physically." Below the User Story, you also paste the Acceptance Criteria from the AC document.

Level 3: Sub-task. This is where the BA breaks down the Story into technical tasks. The developer does not decide what sub-tasks to create. The BA creates the sub-tasks based on the FRD and the System Design. This is important. The BA reads the FRD and the System Design, and from that information, creates specific tasks. For example, sub-task 1: "Create database table: returns (columns: return_id, order_id, customer_id, status, created_at, updated_at)." Sub-task 2: "Implement REST API POST /api/v1/returns (see FRD FR-001, OpenAPI spec)." Sub-task 3: "Build frontend form: return request page with product selection, reason dropdown, photo upload." Sub-task 4: "Implement email notification on successful return submission." Each sub-task references the specific document section that the developer needs to read.

Level 4: Bug. Bugs are created during testing. They are not planned tasks. They get their own issue type. The bug title describes the problem, not the fix. For example: "Return request form crashes when photo exceeds 5 MB" is a good bug title. "Fix photo upload validation" is a bad bug title because it assumes the fix.

Now, there is a standard format for writing Jira task descriptions. Every Jira task, whether it is a Story or a sub-task, should contain five elements.

Element 1: Title. Short and specific. The developer should read the title and immediately know what this task is about. Bad title: "Work on returns." Good title: "Create returns database table with columns for order, customer, items, and status."

Element 2: Description. This is the most important part. The description tells the developer everything they need to know to do the work without calling you. It contains three sections. Context: one or two sentences explaining why this task exists. Reference: links to the FRD section, API spec, or design document that the developer needs to read. Acceptance Criteria: what does "done" look like for this specific task.
Element 3: Reference Links. This is critical. Every Jira task must have links to the relevant documents. Link to the BRD requirement. Link to the FRD section. Link to the API endpoint in Swagger. Link to the test cases in Postman. If the developer has a question, they should be able to click a link and find the answer in the document, not send you a message.
Element 4: Assignee. The BA assigns sub-tasks to the appropriate developer. The backend sub-tasks go to the backend developer. The frontend sub-tasks go to the frontend developer. If you are not sure who should do it, assign it to the lead developer and let them distribute.
Element 5: Sprint and Story Points. The BA does not estimate story points alone. Story points are estimated during Sprint Planning by the development team. The BA presents the task, the team discusses it, and the team assigns the points. The BA only suggests the priority.
Let me give you one complete example of how a Jira Story looks in practice.
Title: "Implement return eligibility check functionality"
Description:
"Context: As part of the Return Management System, customers must be able to check if their order is eligible for return before submitting a return request. This is US-002 from the User Stories document.
User Story: As a registered customer, I want the system to automatically check if my order is eligible for return, so that I know whether I can submit a return request.
Acceptance Criteria:
AC-006: System checks delivery date is within 14 days.
AC-007: System checks item category is returnable.
AC-008: System checks customer has not exceeded 3 returns per month.
AC-009: System displays ineligible reason when any check fails.
AC-010: System shows estimated refund amount when all checks pass.
References:
- FRD: FR-002 (Return Eligibility Check)
- API Spec: GET /api/v1/returns/eligibility
- BRD: BR-002 (Automated Eligibility Check)
- Postman Tests: Collection 'Eligibility Check Tests'"
Assignee: Tural (Backend Developer)
Sprint: Sprint 1
Story Points: 5 (estimated by team)
And under this Story, the sub-tasks would be:
1. "Create eligibility_rules database table" - assigned to backend
2. "Implement eligibility check service class" - assigned to backend
3. "Build eligibility check API endpoint" - assigned to backend
4. "Create eligibility result display component" - assigned to frontend
5. "Write unit tests for eligibility rules" - assigned to QA
So to answer your question: Jira task names do not need to match document names. The Jira task name describes the technical work. But the Jira task description must reference the document

sections. This is the connection. The document is the source of truth. The Jira task is the execution tool. They are connected through references, not through matching names.
One last important point. Never create a Jira task that says "implement BR-001." This tells the developer nothing. The developer should not have to read the BRD to understand what to do. The Jira task description should contain everything the developer needs. The BRD link is there for additional context, not as the primary source of information for the task. The BA is the translator. You read the BRD, you understand it, and you write the Jira task in a way the developer can immediately start working.

Terms of Reference (ToR) and Service Design Document (SDD) - LMAS Project

Q42 In a BA project, you already have a BRD that describes the business requirements and scope. Why do you need a separate Terms of Reference document? What does the ToR cover that the BRD does not?

This is a very practical question, and the answer explains a common confusion. The BRD and the ToR serve completely different purposes, and you need both.

The BRD answers the question: "What does the business need?" It describes the business goals, the stakeholder requirements, the functional and non-functional requirements, and the success criteria. The BRD is written from the business perspective. When you read a BRD, you understand what problem we are solving and what the expected outcome looks like.

The ToR answers a different question: "How will this project be organized and governed?" The ToR defines the project boundaries, who is responsible for what, what the timeline is, what the governance structure looks like, and what the constraints are. The ToR is written from the project management perspective.

Let me give you a concrete example. In the LMAS project for the State Employment Agency, the BRD says: "Citizens must be able to submit employment service applications through digital channels." This is a business requirement. The ToR says: "The BA Lead is responsible for writing the FRD by the end of Phase 2, and the Product Owner must approve it before development starts." This is a project governance rule.

Here is a simple way to think about it. The BRD defines the "what" and the "why." The ToR defines the "who, when and how." Without a BRD, the team does not know what to build. Without a ToR, the team does not know who decides what, when things are due, and how decisions are made.

In government projects like LMAS, the ToR is especially important because multiple government organizations are involved. The ToR makes it clear which agency provides data, who approves requirements, who signs off on deliverables and how disagreements are resolved. Without this clarity, a multi-agency project can easily get stuck.

Q43 What is the correct structure of a Terms of Reference? What sections should it contain and in what order?

A ToR has a standard structure that most government and enterprise projects follow. In my LMAS ToR, I used ten main sections. Let me walk through each one.

Section 1 is Introduction. This includes the document purpose, the background of why the project exists, and definitions of abbreviations. The introduction sets the context for everyone who reads the document. It answers: why are we doing this project?

Section 2 is Project Objectives. This is where you define the strategic goal and the specific measurable objectives. Each objective has an ID, a name and a description. For example, OBJ-05 in my LMAS ToR says: "Reduce average service processing time from 14 calendar days to 5 business days." You also include success criteria with specific targets here.

Section 3 is Project Scope. This is one of the most important sections. It has four parts: in-scope items, out-of-scope items, assumptions and constraints. The in-scope list defines what the project will deliver. The out-of-scope list is equally important because it prevents scope creep. Assumptions are things you believe to be true but have not verified yet. Constraints are limitations you cannot change, like budget limits or regulatory requirements.

Section 4 is Deliverables. This is a complete list of every document, system and report that the project will produce. Each deliverable has an ID, a description, the phase when it is due and the person responsible. In my ToR, I listed 14 deliverables: 8 business analysis deliverables like ToR, BRD, SDD, FRD, SRS, API Specifications, BPMN models and UAT Plan, and 6 technical deliverables like the system architecture, database schema, Telegram bot and web portal.

Section 5 is Project Timeline. This shows the phased approach with milestones. In my LMAS ToR, I defined 4 phases across 12 months with 8 key milestones. Each milestone has a date and a description of what "done" looks like.

Section 6 is Team Structure and Roles. This defines every role in the project, which organization they come from and what their responsibilities are. It also includes the governance structure: steering committee meetings, sprint reviews and stakeholder syncs with defined frequency and participants.

Section 7 is Stakeholder Communication. This is a detailed communication plan that says who you communicate with, through what channel, how often and about what topics. For example, "Steering Committee meets monthly to review project status, risks and budget."

Section 8 is Risk Management. This includes a risk register with probability, impact and mitigation for each risk, plus an escalation path for when risks become critical issues.

Section 9 is Quality Standards. This defines documentation standards, development standards and acceptance criteria for deliverables.

Section 10 is Approval. This includes a signature table where each stakeholder signs to confirm their agreement with the ToR, plus a revision history table.

Q44 You already have a BRD that covers business requirements and scope. You also have a Gap Analysis that covers As-Is and To-Be processes. Why do you need a separate Service Design Document? What does the SDD cover that the BRD and Gap Analysis do not?

This is the most important question about the SDD, because it is the document that people understand the least. Let me explain the difference clearly.

The BRD answers: "What does the business need?" It talks about business goals, stakeholders, scope and requirements. The Gap Analysis answers: "What is wrong with the current process and what should the future process look like?" It shows the As-Is state and the To-Be state at the process level.

The SDD answers a completely different question: "How should the citizen experience this service from start to finish?" The SDD takes the citizen perspective and designs the complete service experience, not just the process or the requirements.

Let me explain with the LMAS example. The BRD says: "The system shall allow citizens to submit employment service applications through digital channels." The Gap Analysis shows the As-Is process: citizen visits office, waits in queue, submits paper form. And the To-Be process: citizen opens Telegram bot, fills digital form, system validates data automatically.

But none of these documents answer the following questions: Who is the citizen using this service? What is their life situation? What are their pain points? What channels do they prefer? What happens at each interaction point? How do multiple government agencies coordinate behind the scenes to deliver this service? What SLA targets should we set? How do we measure if the service is good?

These are exactly the questions the SDD answers. The SDD has several unique sections that do not exist in the BRD or Gap Analysis.

First, User Research and Personas. The SDD starts by defining who the users are. In my LMAS SDD, I created three personas: a young job seeker who is comfortable with technology, a mid-career worker who recently lost their job and needs guidance, and an employer who wants to post vacancies online. The BRD mentions stakeholders, but it does not go deep into user profiles and behavioral patterns.

Second, Service Journey Analysis. The Gap Analysis shows the As-Is and To-Be processes, but the SDD goes deeper. It maps the complete citizen journey including emotional states, pain points at each stage and improvement opportunities. The journey analysis is more detailed because it includes the citizen experience, not just the process steps.

Third, Service Blueprint. This is unique to the SDD. A service blueprint maps the relationship between what the citizen sees and what happens behind the scenes. It has five layers: physical evidence, citizen actions, front-stage activities, back-stage activities and support systems. Neither the BRD nor the Gap Analysis provides this multi-layer view.

Fourth, Service Channels and Touchpoints. The SDD defines which channels will be used (Telegram bot, web portal, ASAN Service centers) and what happens at each touchpoint. The BRD might mention channels, but the SDD designs the complete channel strategy with target volumes for each channel.

Fifth, Multi-Agency Coordination Model. For government services, this is critical. The SDD defines which agencies are involved, what data they provide, what their role is and how they coordinate. The BRD mentions stakeholders but does not design the coordination model.

Sixth, SLA and KPI Framework. The SDD defines specific service level agreements and key performance indicators with measurable targets. For example, SLA-03 says: "Application processing within 5 business days with 90% compliance." KPI-05 says: "Citizen satisfaction score of 4.0 out of 5.0." The BRD might mention success criteria, but the SDD creates a complete monitoring framework.

Seventh, Service Portfolio Alignment. The SDD checks how the new service fits into the existing portfolio of government services and ensures alignment with national digital government standards. This strategic alignment is not covered by the BRD or Gap Analysis.

So in summary: the BRD defines requirements, the Gap Analysis defines process improvements, and the SDD designs the complete citizen service experience including who uses it, how they experience it, what channels deliver it, how agencies coordinate, and how we measure success.

Q45 What is the correct structure of a Service Design Document? What sections should it contain?

A Service Design Document follows a design thinking approach, not a traditional requirements approach. In my LMAS SDD, I used fifteen sections organized into four logical groups.

The first group is Discovery. This includes sections 1 through 4. Section 1 is Executive Summary, which gives a one-page overview of the entire service design for busy decision-makers. Section 2 is Service Vision and Objectives, which defines the long-term vision, strategic objectives and measurable success metrics. Section 3 is Service Context and Stakeholder Analysis, which maps all stakeholders, their roles and how you communicate with each group. Section 4 is User Research and Personas, which documents your research findings, key insights and three to five citizen personas with their needs, behaviors and pain points.

The second group is Design. This includes sections 5 through 9. Section 5 is As-Is Service Journey Analysis, which maps the current citizen journey stage by stage with pain points at each stage. Section 6 is To-Be Service Design, which describes the redesigned service journey with improvements at each stage and design principles that guide all decisions. Section 7 is Service Blueprint, which creates a five-layer view connecting citizen actions to front-stage, back-stage and support systems. Section 8 is Service Channels and Touchpoints, which defines the channel strategy and designs each interaction point. Section 9 is Multi-Agency Coordination Model, which defines how government agencies share data, their roles and the integration architecture.

The third group is Standards. This includes sections 10 through 12. Section 10 is Service Standards, which defines SLA targets for each service stage and KPI indicators with specific measurable targets. Section 11 is Service Portfolio Alignment, which checks how the service fits into the broader government service portfolio and national digital standards. Section 12 is Process Architecture, which defines the three levels of process design: strategic, tactical and operational.

The fourth group is Delivery. This includes sections 13 through 15. Section 13 is Implementation Roadmap, which plans the phased delivery approach with milestones and dependencies. Section 14 is Risks and Dependencies, which includes a risk register with mitigation strategies. Section 15 is Glossary, which defines all key terms used in the document.

One important thing to note: the SDD is not a technical document. It does not contain database schemas, API endpoints or code-level specifications. Those belong in the SRS and API Specification. The SDD is a design document that focuses on the citizen experience and service delivery model.

Q46 You prepared both a ToR and an SDD for the LMAS project. How do these two documents relate to each other? Which one comes first and how do they connect?

The ToR comes first, and the SDD comes second. They have a parent-child relationship where the ToR sets the boundaries and the SDD fills in the design details.

Think of it like building a house. The ToR is like the building permit and the project plan. It says: we are building a three-bedroom house on this plot of land, with this budget, this timeline and these people responsible. It defines what you are allowed to build and how the project will be managed.

The SDD is like the architectural design. It says: the living room will be here, the kitchen will face east for morning sunlight, the entrance will have these features for wheelchair accessibility and the garden will be designed for family gatherings. It designs the experience of living in the house.

In the LMAS project, the ToR says: "OBJ-01: Enable citizens to submit employment service applications through digital channels." This is a project objective with scope and timeline. The SDD then designs how this actually works: which channels, what the citizen journey looks like, what touchpoints the citizen interacts with, what happens behind the scenes and how success is measured.

The ToR also defines the deliverables list. In my ToR, deliverable D-03 is the SDD itself. So the ToR creates the requirement to produce the SDD, defines who is responsible and sets the deadline. The SDD then fulfills this requirement.

Another connection is the scope. The ToR defines what is in-scope and out-of-scope. The SDD must stay within these boundaries. For example, my ToR says that a mobile native application is out of scope for Phase 1. So the SDD designs the service for Telegram bot and web portal only, not for a mobile app. If the SDD started designing a mobile app experience, that would be a scope violation.

The risk register in the ToR also connects to the SDD. For example, ToR risk R-03 says: "Low citizen adoption of digital channels." The SDD addresses this risk by designing a simplified onboarding experience in the citizen journey and by defining ASAN Service centers as a fallback channel for citizens who cannot use digital channels.

So the relationship is: ToR defines the project frame and governance. SDD designs the service within that frame. ToR is the first document you write. SDD comes after, and it must respect the scope, timeline and constraints defined in the ToR.

Q47 You have prepared many BA documents: ToR, BRD, SDD, FRD, SRS, User Stories, API Specifications, BPMN models and UAT Plan. In what order do you prepare them and why?

There is a logical order, but in practice some documents overlap. Let me explain the ideal order and then how it works in reality.

The ideal order follows a waterfall approach within each phase. Phase 1 starts with the ToR, because you cannot start a project without defining the scope, objectives, governance and timeline. The ToR gets approved by the steering committee. Then you write the BRD, which defines the business requirements in detail. Then you write the SDD, which designs the service experience. Then you create BPMN models for the As-Is and To-Be processes. All of these happen in Phase 1.

Phase 2 is when you write the FRD, which converts business requirements into detailed functional requirements with user stories and acceptance criteria. Then you write the SRS, which adds technical system requirements. Then you write the API Specifications for all integrations. These happen in Phase 2.

Phase 3 is when you prepare the UAT Plan and execute testing with stakeholders.

So the ideal order is: ToR, BRD, SDD, BPMN, FRD, SRS, API Spec, UAT Plan.

But in reality, some documents are written in parallel. For example, I usually start the BRD and the SDD at the same time because they inform each other. When I write a BRD requirement like "citizens must be able to track application status," this influences the SDD service journey design. And when I design the citizen journey in the SDD, I often discover new requirements that need to go into the BRD. So I work on both documents and update them as I go.

The same is true for the FRD and SRS. The FRD defines the functional requirements, and the SRS adds the technical details. These two documents are closely connected, and I usually work on them together.

One important rule I follow: the ToR must be completed and approved first before any other document. The ToR defines the project boundaries, and if you start writing the BRD without an approved ToR, you risk writing requirements for things that are out of scope or missing things that are in scope.

Another rule: the UAT Plan is always the last document. You cannot plan testing until all requirements are defined and documented.

Q48 The Innovation Agency uses Service Design methodology. How does your SDD document reflect Service Design methodology, and what Service Design tools and techniques did you use?

Service Design is a methodology that puts the citizen at the center of the design process. Instead of designing services from the institution perspective, you design them from the citizen perspective. My SDD uses several Service Design tools and techniques to apply this methodology.

The first tool is User Research. In Service Design, you always start with research before design. In my SDD, section 4 documents the user research I conducted: 12 stakeholder interviews and analysis of 200+ citizen feedback records. This research identified six key findings about citizen pain points, which directly influenced all design decisions.

The second tool is Personas. Personas are fictional characters that represent real user groups. They help the design team think about real people instead of abstract requirements. In my SDD, I created three personas based on the research data: a young job seeker, a mid-career worker and an employer. Each persona has specific needs, behaviors and pain points that the service design must address.

The third tool is Service Journey Mapping. This is one of the most important Service Design tools. A journey map shows every step a citizen takes from the moment they become aware of a service to the moment they receive the result. In my SDD, sections 5 and 6 contain the As-Is and To-Be journey maps. The As-Is map shows seven stages of the current process with pain points at each stage. The To-Be map shows the redesigned seven stages with specific improvements.

The fourth tool is Service Blueprinting. A service blueprint is like an x-ray of the service. It shows five layers: what the citizen sees (physical evidence), what the citizen does (actions), what happens in front of the citizen (front-stage), what happens behind the scenes (back-stage) and what systems support everything (support systems). In my SDD, section 7 contains the service blueprint that maps all five layers.

The fifth tool is Life-Event Based Design. This is a Service Design approach where services are organized around key life events instead of government departments. Instead of saying "these are the services the employment agency provides," you say "when a citizen loses their job, these are all the services they need." In my SDD, I applied this approach by designing the service around the life event of unemployment rather than around agency departments.

The sixth tool is Multi-Channel Design. Citizens should be able to access services through multiple channels: Telegram bot for quick interactions, web portal for complex tasks, ASAN centers for exceptional cases. My SDD section 8 designs the channel strategy with target volumes for each channel.

The seventh tool is SLA and KPI Design. Service Design is not only about designing the service but also about measuring its performance. My SDD section 10 defines six SLAs and seven KPIs with specific measurable targets. This is the "how do we know the design works" part of Service Design.

So my SDD is not just a document. It is the output of applying Service Design methodology to a real government service. Each section uses a specific Service Design tool to ensure the final service is citizen-centered, measurable and aligned across multiple agencies.

Q49 The Innovation Agency wants to consolidate government services from individual agency applications into the MyGov platform. What does service consolidation mean for a Business Analyst, and what are the main challenges?

Service consolidation means bringing services that are currently delivered through separate agency platforms, such as the Azersu mobile app or the Ministry of Culture portal, into one unified platform called MyGov. Instead of a citizen downloading ten different apps for ten different services, they use one platform for all government services.

This concept is not new. The United Kingdom did this with GOV.UK, consolidating over 300 government websites into one portal. Estonia did it with e-Estonia, bringing all public services

under a single digital platform. India did it with the UMANG app, integrating services from 120-plus central and state government departments. In each case, the BA played a central role in analyzing, planning and executing the consolidation.

For a Business Analyst, service consolidation involves several major activities. First, service inventory: you must catalog every service that each government agency currently provides digitally. This means meeting with every agency, understanding their service list, and documenting each service with its current process, delivery channel, backend system, data sources and user volume. Many agencies do not have a complete or up-to-date service catalog, so this step alone can take months.

Second, prioritization: you cannot migrate 500 services at once. You need a prioritization framework based on factors like citizen demand, technical complexity, agency readiness and strategic importance. High-volume, high-impact services with technically ready agencies should come first. This requires close coordination with agency leadership and the Innovation Agency steering committee.

Third, integration design: each agency has different backend systems, different data formats and different levels of technical maturity. The BA must define the interoperability requirements between MyGov and each agency system, including API standards, data exchange protocols, security requirements and authentication methods. Some agencies have modern REST APIs, while others still use legacy systems with no API at all.

The main challenges are both technical and organizational. On the technical side, legacy systems without APIs, inconsistent data models across agencies, and security concerns about cross-agency data exchange are the biggest obstacles. On the organizational side, agency resistance is the hardest challenge. Many agencies see their digital platforms as a symbol of their institutional identity and independence. Convincing them to migrate their services to MyGov requires strong political support, clear communication of benefits and sometimes formal government mandates.

Q50 What is an interoperability layer, and why is it important for the MyGov platform?

An interoperability layer is a technical middleware that enables different systems to communicate with each other. Think of it as a universal translator. When Agency A speaks one technical language and Agency B speaks a different language, the interoperability layer sits between them and translates so they can exchange data seamlessly.

In the context of MyGov, the interoperability layer is the component that connects the MyGov front-end platform to the backend systems of all government agencies. When a citizen submits a service request through MyGov, the interoperability layer routes this request to the correct agency system, translates the data format, handles authentication and returns the response back to MyGov.

The most famous example of an interoperability layer in the world is X-Road in Estonia. X-Road connects over 900 organizations and handles more than one billion data exchange transactions per year. Every time a citizen accesses a government service in Estonia, X-Road coordinates the data exchange between the relevant agencies behind the scenes. The citizen does not see any of this complexity; they simply see a fast, seamless service experience.

As a Business Analyst, my role in the interoperability layer is not to build it, because that is the job of the technical architects and developers. My role is to define what the interoperability layer must do. This means documenting the data exchange requirements for each agency, defining the API standards and message formats, specifying the security and authentication rules, and describing the error handling and fallback procedures. I did similar work at the Central Bank when I coordinated the integration of ten-plus government organizations into the Government Payment

Portal, where I defined data exchange requirements and integration specifications for each agency and designed the interoperability layer for cross-system data exchange.

Q51 What does a registry-based approach mean in digital government, and how is it different from simply calling an API?

A registry-based approach means that government data is stored in centralized, authoritative databases called registries, and all government agencies access this data through the registries instead of maintaining their own copies. The registry is the single source of truth.

For example, in Azerbaijan, the citizen registry holds basic information about every citizen: name, address, family status and identification number. When a citizen applies for a service at any agency, the agency does not ask the citizen to provide copies of their ID or proof of address. Instead, the agency system queries the citizen registry through the interoperability layer and retrieves the data automatically.

This is fundamentally different from a simple API call. With a standard API, System A calls System B to get some data. System B owns the data and System A just borrows it. With a registry-based approach, the registry is the authoritative owner of that data, and no other system stores a permanent copy. Every time any agency needs that data, they access the registry in real time.

There are several important benefits. First, data consistency: because there is only one authoritative copy, there are no conflicts between different agency databases. Second, the once-only principle: citizens do not need to submit the same information multiple times to different agencies. Third, data governance: it is clear who owns the data, who can access it and under what conditions.

In practice, I worked with registry-based data exchange at ASAN Service, where citizen data from multiple government registries was accessed to deliver services without requiring citizens to bring paper documents. This experience is directly relevant to the MyGov platform, where the same registry-based approach will be used at a larger scale across all government services.

Q52 What is the once-only principle in digital government, and how does it affect how you design services for MyGov?

The once-only principle is a fundamental rule of digital government that says: citizens and businesses should only have to provide the same information to the government once. If you already submitted your address to one agency, no other agency should ask you for your address again. They should retrieve it from the relevant registry or database.

This principle is actually a legal requirement in the European Union, under the EU Digital Single Market regulations. Estonia was one of the first countries to implement it fully. In Estonia, when you submit any information to any government agency, that information is automatically available to all other agencies through the X-Road interoperability layer. Citizens never have to fill in the same data twice.

For the MyGov platform, the once-only principle has a direct impact on how I design services. When I write the SDD for a service migration to MyGov, I must analyze every data field in the current service form and ask: is this data already available in a government registry? If yes, then the MyGov service should not ask the citizen to enter it manually. Instead, the system should retrieve it automatically from the registry.

For example, if a citizen applies for a service through MyGov and the form asks for their full name, date of birth and address, I need to check whether this data exists in the citizen registry. If it does, I design the service so that these fields are pre-populated from the registry and the citizen only needs to verify them, not type them. This reduces form filling time, reduces errors and improves the citizen experience.

There are practical challenges, of course. Not all data exists in registries. Some agencies still rely on paper-based records. Some registries are not connected to the interoperability layer. And there are data privacy and consent concerns: citizens must be informed about which agencies are accessing their data and must give their consent. As a BA, I need to document all these constraints in the SDD and define how the service handles cases where registry data is not available.

Q53 When migrating hundreds of services from individual agency platforms to MyGov, how do you decide which services to migrate first?

This is one of the most important decisions in a service consolidation project, and the answer is: you need a structured prioritization framework. You cannot migrate services randomly or based on which agency complains the loudest. You need objective criteria.

I use a multi-factor prioritization matrix. The first factor is citizen demand, which means how many citizens use this service and how frequently. A service that ten thousand citizens use every day should clearly be migrated before a service that fifty citizens use once a year. I can measure this using analytics data from the current agency platforms.

The second factor is citizen impact, which means how much the citizen experience improves after migration. If a service currently requires three physical office visits and ten paper documents, migrating it to MyGov will have a huge impact. If a service is already available online and works well, the impact of migration is smaller, even if the volume is high.

The third factor is technical complexity. Some services are simple: one form, one database, one approval step. These can be migrated quickly. Other services require integration with five different backend systems, real-time data exchange and complex business rules. These take much longer. I need to balance the value of migrating a service against the cost and time required.

The fourth factor is agency readiness. Some agencies have modern IT systems, well-documented APIs and motivated teams. They are ready to integrate with MyGov quickly. Other agencies have legacy systems, no APIs and limited IT capacity. They need more time and support. I assess agency readiness through a structured evaluation that covers technical infrastructure, data quality, documentation and organizational willingness.

The fifth factor is strategic alignment. The Innovation Agency may have political priorities or national strategy goals that influence the order. For example, if the President announced that unemployment services must be digital by a certain date, that service gets higher priority regardless of other factors.

In practice, I would create a prioritization matrix with all services listed, score each service against these five factors, and recommend a phased migration plan. Phase 1 would include high-volume, high-impact, low-complexity services from ready agencies. Phase 2 would add medium-complexity services. Phase 3 would tackle the most complex integrations. This is the same phased approach I used in the GPP project at the Central Bank, where I coordinated the integration of ten-plus organizations into a single platform in multiple phases.

Q54 What is the difference between platform thinking and project thinking, and why does it matter for MyGov?

This is an important distinction. Project thinking means you focus on delivering one specific service or one specific feature. You define the requirements, you build it, you test it and you deliver it. Your success is measured by whether that one service works correctly. This is how most BA work has traditionally been done.

Platform thinking means you focus on building a foundation that can support many services, now and in the future. You are not building one service; you are building the infrastructure, standards

and reusable components that allow hundreds of services to be added efficiently over time. Your success is measured by how easily new services can be added to the platform.
Let me give you a concrete example. In project thinking, if you need to build an online application form for a specific service, you design the form, build it and deploy it. Done. In platform thinking, you design a generic form engine that can generate any type of government service form. The first service uses this engine, the second service uses the same engine, and the fiftieth service also uses the same engine. You invest more time in the beginning, but each additional service becomes cheaper and faster to deploy.
The same logic applies to other components. Instead of building a payment integration for one service, you build a shared payment gateway that all services use. Instead of building a notification system for one agency, you build a central notification service that any agency can connect to. Instead of building user authentication for each service, you build a single sign-on system that works across all services on MyGov.
For a Business Analyst, platform thinking changes how I write requirements. Instead of writing requirements that are specific to one service, I try to write requirements that define reusable patterns and standards. For example, instead of saying 'the employment service application form shall have five fields,' I say 'all service application forms on the MyGov platform shall follow the standard form design template defined in the MyGov Design System, with agency-specific fields defined in the service configuration.'
This approach requires more upfront analysis and more abstract thinking, but it pays off enormously when you are dealing with hundreds of services. It ensures consistency across services, reduces development costs over time and makes the platform easier to maintain and scale.
Q55 In a service consolidation project, how do you handle agency resistance to migrating their services to MyGov?
Agency resistance is the single biggest risk in any service consolidation project, and it is completely understandable. When an agency has spent years building their digital platform, training their team and establishing their online presence, being told to migrate everything to a shared platform feels like losing control, losing visibility and losing institutional identity.
The first and most important principle is that resistance cannot be overcome by technical arguments alone. You cannot convince an agency director to migrate by showing them API documentation and system architecture diagrams. You need to speak their language and address their real concerns.
I use a stakeholder engagement approach that has four steps. The first step is listening. Before I propose anything, I schedule meetings with agency leadership and their IT teams. My goal is to understand their perspective: what do they like about their current platform? What are they proud of? What are they worried about losing? Often, their concerns are very practical. They worry about response times for their citizens. They worry about data security. They worry that their team will become irrelevant.
The second step is framing the benefits correctly. I do not say 'you must migrate because the government decided so.' Instead, I frame it as a win for the agency. I explain that by joining MyGov, their services will reach more citizens because MyGov has higher traffic than their individual app. I explain that the Innovation Agency will handle platform maintenance, which reduces their IT budget. I explain that their agency brand and service quality will still be visible within the MyGov platform.
The third step is proposing a phased transition that minimizes disruption. I never propose a hard cutover where their app is shut down on Monday and MyGov takes over on Tuesday. Instead, I

propose a parallel period where both the agency app and the MyGov service run simultaneously. Citizens can choose which one to use. Over time, we gradually redirect traffic to MyGov. This gives the agency time to adapt and gives their citizens time to get used to the new channel.
The fourth step is escalation support. Sometimes, despite all efforts, an agency simply refuses to cooperate. In these cases, I document the situation, the impact on the consolidation timeline and the risks, and escalate to the Innovation Agency leadership or the relevant government authority. Political mandate is sometimes necessary, but it should always be the last resort, not the first approach.
I have direct experience with multi-agency coordination from my work at the Central Bank, where I coordinated the integration of ten-plus government organizations into the Government Payment Portal. Each organization had its own systems, its own priorities and its own concerns. My approach was always to listen first, then find the mutual benefit, then plan the transition carefully.
Q56 What is a common data model, and why is it needed when multiple agencies share data through MyGov?
A common data model is a standardized way of organizing and representing data that all agencies agree to use when they exchange information through the MyGov platform. It is like a shared language for data.
Let me explain the problem first. Every government agency has its own way of storing and naming data. One agency might call a citizen address field 'citizen_address', another might call it 'residence_location', and a third might split it into 'street_name', 'house_number' and 'city' as separate fields. When these agencies need to exchange data, these differences cause confusion, errors and integration failures.
The common data model solves this problem by defining standard field names, standard data types, standard formats and standard code lists that all agencies must follow when exchanging data through MyGov. For example, the common data model defines that every citizen record must have a field called 'address_line_1' in text format, a field called 'postal_code' in five-digit format, and so on.
For a Business Analyst, creating and maintaining the common data model is a significant part of the job. I need to analyze the data structures of each agency, identify the common elements, resolve conflicts and agree on a standard format with all stakeholders. This requires many workshops with agency IT teams and data architects.
The common data model is especially important for MyGov because the platform needs to display data from multiple agencies in a unified way to the citizen. When a citizen logs into MyGov and sees their profile, they expect to see their information correctly formatted and consistent, regardless of which agency provided the data. The common data model makes this possible.
My experience at ASAN Service and the Central Bank prepared me well for this work. At the Government Payment Portal, I defined data exchange requirements and integration specifications for each agency, which required understanding each agency's data model and creating a standardized format for cross-system data exchange. This is essentially what a common data model does at a larger scale for the MyGov platform.